esprit

Se former autrement

2022 - 2023

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : INFORMATION TECHNOLOGY

TITLE

By: *(full name)*

Academic supervisor: *(full name)*

Corporate Internship Supervisor: *(full name)*

LOGO ENTREPRISE

cdio

Délivrée par: Cti

EUR-ACE

CONFÉRENCE DES GRANDES ÉCOLES

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : ……..

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : ……..

Cachet & Signature / Stamp & Signature

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : ……..

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

Dedication

To my father, ***Anis Gharbi****, and my mother,* ***Sana Menzli****,*
for their love, sacrifice, and unwavering support.

To my grandparents, ***Mohamed Habib Menzli*** *and* ***Hayet****,*
for their blessings and constant encouragement.

To my uncles, aunts, and the wider family,
for their warmth and support.

To my little sister, ***Yara****.*

To my friends, ***Mohamed Yassine Ben Zekri****,* ***Aziz Ben Lamine****,* ***Saif Yaakoubi****, and* ***Ons Matmati*** *for their encouragement.*

I dedicate this work to all of you.

Acknowledgments

I would like to thank my academic supervisor, **Saoussen Lakhdhar**, at ESPRIT, for her guidance and feedback throughout this work.

I am also grateful to **Oumaima Aounallah** and the wider Coral-io team for their technical guidance and for granting the autonomy that shaped this project's research-oriented direction.

Finally, I thank the members of the jury for their time and attention in evaluating this work.

Contents

Acknowledgments ii

List of Figures vi

List of Tables viii

List of Abbreviations xi

1 General Introduction 1
- 1.1 General Context . . . 1
- 1.2 Host Organization . . . 1
  - 1.2.1 Presentation of Coral-io . . . 1
  - 1.2.2 Internship Framework . . . 2
- 1.3 Problem Statement . . . 2
  - 1.3.1 Limitations of Existing Approaches . . . 2
  - 1.3.2 The Core Gap . . . 3
  - 1.3.3 Formal Problem Statement . . . 3
- 1.4 Proposed Solution . . . 3
- 1.5 Project Objectives and Scope . . . 4
  - 1.5.1 Objectives . . . 4
  - 1.5.2 Scope . . . 5
- 1.6 Requirements Specification . . . 6
  - 1.6.1 Functional Requirements . . . 6
  - 1.6.2 Non-Functional Requirements . . . 6
- 1.7 Project Methodology and Work Planning . . . 7
  - 1.7.1 Choice of Methodology: CRISP-DM over Sprint-Based Agile . . . 7
  - 1.7.2 Development Methodology: CRISP-DM . . . 8
  - 1.7.3 Development Phases . . . 8
  - 1.7.4 Rationale for the Approach . . . 9

2 State of the Art 11
- 2.1 Introduction . . . 11
- 2.2 Algorithmic Trading: Evolution and Challenges . . . 11
- 2.3 Market Regime Detection . . . 11
  - 2.3.1 Rule-Based Approaches . . . 11
  - 2.3.2 Statistical Approaches: Hidden Markov Models and Clustering . . . 12
  - 2.3.3 Deep Learning Approaches . . . 12
- 2.4 LLM Agents in Finance . . . 12
  - 2.4.1 Single- and Multi-Agent LLM Trading Systems . . . 12

2.4.2 Multi-Agent Debate as a Design Pattern . . . 13
2.4.3 Limitations of Existing Approaches . . . 13
2.5 Reinforcement Learning for Trading . . . 13
2.5.1 Early Approaches: DQN and Discrete Action Spaces . . . 13
2.5.2 PPO and Continuous Action Spaces . . . 13
2.5.3 Curriculum Learning in Finance . . . 14
2.6 Positioning of MARTA . . . 14
2.7 Summary . . . 15

3 Methodology **16**
3.1 Overview . . . 16
3.2 System Design and UML Models . . . 16
3.2.1 Use Case Model . . . 16
3.2.2 Class Model . . . 18
3.2.3 Sequence Model . . . 18
3.2.4 Deployment Model . . . 19
3.3 Data and Feature Engineering . . . 20
3.3.1 Data Sources and Coverage . . . 20
3.3.2 Pipeline Stages . . . 21
3.3.3 Train / Validation / Test Split . . . 21
3.4 Regime Encoder . . . 22
3.4.1 Design Objective . . . 22
3.4.2 Architecture . . . 23
3.4.3 Contrastive Training . . . 23
3.4.4 Ongoing Refinement: Pooling and Pair Construction . . . 24
3.5 Regime Detection . . . 24
3.5.1 Design Objective . . . 24
3.5.2 Dimensionality Reduction . . . 24
3.5.3 Model Selection . . . 25
3.5.4 Final Fit and Regime Stream Generation . . . 25
3.5.5 Engine Correctness Audit . . . 26
3.5.6 Asset Panel Revision . . . 27
3.5.7 Empirical State-Count Validation at Scale . . . 28
3.5.8 Synthetic Ground-Truth Validation . . . 28
3.6 Multi-Agent Deliberation System . . . 29
3.6.1 Design Objective . . . 29
3.6.2 Orchestration and LLM Backend . . . 29
3.6.3 Regime Context . . . 30
3.6.4 Agent Roster . . . 30
3.6.5 Debate Flow . . . 30
3.6.6 Risk Limit Computation . . . 30
3.6.7 Fund Manager Synthesis . . . 31
3.7 RL Execution Agent . . . 32
3.7.1 Design Objective . . . 32
3.7.2 MDP Formulation . . . 32
3.7.3 Policy Network . . . 33
3.7.4 Curriculum Training . . . 34
3.7.5 Evaluation Determinism . . . 34

3.8 Evaluation Methodology . . . 35
3.8.1 Metrics . . . 35
3.8.2 Ablation Design . . . 35
3.8.3 Statistical Testing . . . 35
3.8.4 In-Sample Regime Behavior Diagnostic . . . 35
3.9 Dashboard and Live System . . . 35
3.9.1 Design Objective . . . 35
3.9.2 Architecture . . . 36
3.9.3 Live Regime Lookup . . . 36
3.9.4 Live Feature Bridge . . . 37
3.9.5 Multi-Asset Debate Coverage . . . 37
3.9.6 User-Configurable Strategy Backtesting . . . 37
3.9.7 Auditability . . . 38

4 Results **42**
4.1 Regime Detection Results . . . 42
4.1.1 Statistical Validity Across Candidate Configurations . . . 42
4.1.2 Regime Characterization . . . 43
4.1.3 Downstream Value: Position Sizing and Directional Conditioning . . . 44
4.1.4 Regime Label Directional Fidelity: A Leg-Level Check . . . 45
4.1.5 Consequence: Removing Directional Language from Regime Descriptions (2026-09-01) . . . 46
4.1.6 Synthetic Recovery Summary . . . 47
4.2 Multi-Agent Deliberation Results . . . 48
4.2.1 Validation Battery: What the System Actually Does . . . 48
4.2.2 Failure Modes Found, and Their Resolution . . . 48
4.2.3 Relationship to the Reported RL Policy . . . 50
4.3 RL Execution Results . . . 50
4.3.1 Controlled Ablation on the Crypto-Only Panel . . . 50
4.3.2 Predictability Ceiling Check . . . 51
4.3.3 Final Decision on the RL Layer . . . 52
4.3.4 Native Weekly-Decision-Cadence Follow-Up (2026-08-31) . . . 52

5 Conclusion **57**
5.1 Regime Detection: Findings . . . 57
5.1.1 What Was Found . . . 57
5.1.2 Interpretation . . . 58
5.2 Multi-Agent Deliberation: Findings . . . 58
5.3 RL Execution: Findings . . . 59
5.4 Overall Verdict . . . 60
5.5 Limitations and Open Items . . . 61

A Appendix **62**
A.1 Synthetic Regime Ground-Truth Validation: Full Derivation . . . 62
A.2 Implementation Note: A Multiprocessing Deadlock . . . 64

Résumé **66**

Abstract **67**

List of Figures

1.1 MARTA's five-layer pipeline, grouped according to the two-part problem statement of Section 1.3.3. Each layer is detailed in Chapter 3. 4
1.2 Project planning Gantt chart 10

3.1 Use case diagram: actors, use cases, and <<include>>/<<uses>> relationships across the five layers. 17
3.2 Class diagram: agent hierarchy, debate data classes, and RL environment/policy classes. 18
3.3 Sequence diagram: one hourly decision cycle, from a new OHLCV bar to a placed paper order and dispatched alert. 19
3.4 Deployment diagram: Docker containers, mounted volumes, the host-process LLM, and external service dependencies. 20
3.5 Contrastive pretraining of the regime encoder: two augmented views of the same window are encoded with shared weights and pulled together in embedding space, while other windows in the mini-batch act as negatives pushed apart by the NT-Xent loss. 23
3.6 Regime detection pipeline: engineered features → BIC-guided state-count search → best-of-5-seed Gaussian HMM fit → regime probability stream, consumed by Layers 3 and 4. 26
3.7 Multi-agent deliberation flow: Round 1 (three LLM directional agents plus the deterministic Risk agent), the conditional Contrarian check and Round 2 rebuttals, and the deterministic Fund Manager synthesis producing the final decision. 31
3.8 RL execution agent: the shared policy trunk feeds a Categorical direction head and a Beta position-fraction head, whose combined output is executed by the environment and scored by the CVaR-normalized reward, producing the next observation. 33

4.1 Per-regime historical Sharpe (bars, left axis) and annualized volatility (line, right axis), six-state five-asset 22-column configuration. Time-share annotations below each bar show the fraction of pooled 2018–2025 hours spent in that regime. 45
4.2 Absolute Sharpe ratio per ablation row against the buy-and-hold reference (dashed line, Sharpe 1.498). Error bars show the 95% block-bootstrap confidence interval on the Sharpe delta, re-centered onto the absolute Sharpe axis; bar color follows the significance verdict of Table 4.9. 51

4.3 BTC-only weekly-cadence grid: the pre-registered checkpoint's Δ Sharpe vs. buy-hold on 2024 validation (blue) against the same checkpoint's Δ on 2025 test (red/green), from Table 4.11. Diamonds show the individual-seed 2024 Δ values of Table 4.10 as context for the spread the pre-registered checkpoint was drawn from. Conditions A and B flip sign between the two years; C and D do not. 53
4.4 Crypto5 panel extension, 2025-only Δ Sharpe vs. buy-hold per asset and condition (Table 4.12). ETH is the one asset where all four conditions, including both the real regime condition (C) and its noise control (D), lose to buy-hold. 55

List of Tables

1.1 CRISP-DM phases 8
1.2 MARTA development phases 9

2.1 Comparison of MARTA against representative prior systems 14

3.1 The 14 features per asset used throughout the pipeline 22
3.2 Regime encoder architecture (`TFTRegimeEncoder`) 39
3.3 Encoder training hyperparameters 39
3.4 The six agents of Layer 3 (five LLM-based deliberation agents plus one deterministic risk-limit agent) 40
3.5 PPO training configuration 40
3.6 Ablation variants 41

4.1 Seed stability and distinctness across all six configurations tested. Seed ARI is the mean pairwise Adjusted Rand Index across five independent EM restarts (Section 3.5.5). `close_return` / `realized_vol_raw` distinctness report the outcome of a per-asset ANOVA across states on that quantity. . 43
4.2 Next-hour directional predictive signal (AUC, logistic regression, walk-forward test split) for the same six configurations. 43
4.3 Per-regime fitted statistics, six-state five-asset 22-column configuration (pooled 2018–2025 training data). Sharpe, annualized return, and annualized volatility are historical population averages conditional on regime membership, not forward-looking predictions — see Section 4.1.4. 44
4.4 Regime-conditioned volatility-target position sizing versus buy-and-hold and continuous (regime-free) volatility targeting, five-asset equal-weight portfolio. 44
4.5 Directional gating: real regime versus a matched noise control, momentum strategy, five-asset portfolio. 45
4.6 Regime-label dominance by leg direction, five-asset panel, 2021 (111 legs total, zigzag threshold 15%). 46
4.7 Synthetic ground-truth recovery (Adjusted Rand Index) across every mechanism tested (Section 3.5.8). 48
4.8 Multi-agent layer validation battery (20 real Mistral 7B calls, `BTC_USDT` 2024-03-01). 48
4.9 Controlled ablation, crypto-only panel, BTC(+ETH) validation window. Δ vs. buy-hold with 95% block-bootstrap CI (Section 3.8.3). 50

4.10 Native weekly-cadence grid, BTC-only, validation year 2024. Sharpe recomputed with buy-and-hold aggregated to the identical weekly cadence (52 weekly observations); Δ from a block bootstrap (block size 4 weeks, 1000 resamples). Recomputed buy-hold reference: Sharpe 1.571. Full detail: `data/results/weekly_grid_reconciled.json`. 53
4.11 2025 test-set pass, pre-registered checkpoints, one per condition. Δ vs. buy-hold; recomputed buy-hold weekly Sharpe on 2025 is -0.291 (-15.9% return), against $+1.571$ on 2024 — a materially different market year. . . . 54
4.12 2025-only Δ vs. buy-hold, extracted from a single continuous 2024–2025 walk-forward pass per checkpoint, crypto5 panel, 1 seed. 0 of 20 cells significant. 55

List of Abbreviations

Abbreviation	Meaning
RL	Reinforcement Learning
LLM	Large Language Model
HMM	Hidden Markov Model
PPO	Proximal Policy Optimization
MDP	Markov Decision Process
TFT	Temporal Fusion Transformer
InfoNCE	Information Noise-Contrastive Estimation (contrastive loss)
CPC	Contrastive Predictive Coding
LSTM	Long Short-Term Memory
VSN	Variable Selection Network
DQN	Deep Q-Network
SAC	Soft Actor-Critic
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
VaR	Value at Risk
CVaR	Conditional Value at Risk
BIC	Bayesian Information Criterion
EM	Expectation-Maximization
PCA	Principal Component Analysis
ARI	Adjusted Rand Index
ANOVA	Analysis of Variance
AUC	Area Under the (ROC) Curve
ROC	Receiver Operating Characteristic
CI	Confidence Interval
CRISP-DM	Cross-Industry Standard Process for Data Mining
OHLCV	Open, High, Low, Close, Volume
RSI	Relative Strength Index
MACD	Moving Average Convergence Divergence
ADX	Average Directional Index
ATR	Average True Range
BB	Bollinger Bands
API	Application Programming Interface
REST	Representational State Transfer
WS	WebSocket
JSON	JavaScript Object Notation
UML	Unified Modeling Language
CCXT	CryptoCurrency eXchange Trading (library)
GPU	Graphics Processing Unit
CPU	Central Processing Unit

Chapter 1

General Introduction

1.1 General Context

Financial markets process trillions of dollars in transactions daily across equities, currencies, commodities, and digital assets. Over the past two decades, algorithmic trading has evolved from a niche practice used by quantitative hedge funds into the dominant force in global markets, now accounting for the majority of daily trading volume on major exchanges.

The core appeal is clear: algorithms process data at speeds no human can match and are immune to emotional bias. Yet building a consistently profitable system remains one of the hardest problems in applied machine learning. Markets are non-stationary — their statistical properties shift over time. A strategy that excels during a trending bull market may fail entirely in a volatile, mean-reverting environment. This sensitivity to changing market conditions is not a minor detail; it is the central challenge of the field.

Early rule-based systems lacked adaptability. Machine learning, and reinforcement learning in particular, offered a path forward by framing trading as a sequential decision problem. More recently, large language models introduced the ability to reason over unstructured data — news, macroeconomic commentary, sentiment — opening a new dimension in trading research. However, existing systems that combine these approaches share a critical blind spot: none condition their decisions on a *learned* representation of the current market regime. This gap defines the motivation for MARTA.

1.2 Host Organization

1.2.1 Presentation of Coral-io

Coral-io is a technology consulting firm specializing in digital transformation and IT solutions across multiple industries. The company provides comprehensive services that bridge technical expertise with business objectives, helping clients navigate complex technological challenges. Its activity spans five main areas:

- **DevOps Services**: CI/CD pipeline implementation, Infrastructure as Code, containerization and orchestration (Docker, Kubernetes), and test automation.
- **Application Performance Management (APM)**: full-stack observability, real-time monitoring, and bottleneck diagnosis for production software.

- **Third-Party Solution Integration**: as a Micro Focus (OpenText) partner, Coral-io delivers IT Operations Management (ITOM) integration and legacy system modernization.
- **Research & Development**: proof-of-concept development and emerging-technology adoption, sitting at the intersection of technical expertise and applied innovation.
- **Finance and Insurance Consulting**: FinTech implementation, regulatory compliance, and digital transformation for financial services clients.

MARTA was carried out within the R&D and Finance/Insurance Consulting scope of the company's activity, combining applied AI research with a financial-markets use case.

1.2.2 Internship Framework

Unlike a project built around a pre-defined internal need, this final-year project was conducted under a self-directed research framework: Coral-io granted full autonomy in selecting the subject and technical direction of the work.

Within this framework, technical guidance was provided collaboratively by Ms. Oumaima Aounallah and the wider Coral-io team, consulted on an as-needed basis for architectural decisions, tooling questions, and periodic progress reviews rather than through prescriptive task assignment. Academic supervision was provided by Saoussen Lakhdhar at ESPRIT.

This structure placed a greater share of responsibility for scoping, planning, and technical decision-making than is typical of a company-assigned project — a distinction elaborated further in Section 1.7.

1.3 Problem Statement

1.3.1 Limitations of Existing Approaches

Three broad families of automated trading systems exist today, each with a distinct blind spot relevant to MARTA.

Rule-based and classical statistical strategies (moving-average crossovers, mean-reversion bands, momentum filters) are transparent and computationally cheap, but their parameters are fixed at design time. When the market regime shifts — from trending to ranging, from low to high volatility — these fixed parameters no longer match the underlying data-generating process, and performance degrades until the strategy is manually re-tuned.

Pure reinforcement learning agents address this rigidity by learning a policy directly from historical price action, allowing some adaptation to changing dynamics. However, standard RL formulations treat the market as a single, undifferentiated environment: the agent is not explicitly told which regime it is currently operating in, and must infer this implicitly through its state representation. In practice, this means the policy tends to average its behavior across regimes rather than specializing for each one, and offers no human-interpretable explanation for why a given action was chosen.

LLM-based trading agents, a more recent development, exploit large language models' ability to reason over unstructured text — news headlines, macroeconomic reports, analyst commentary — to produce qualitative market assessments. This adds a

reasoning layer absent from purely numerical approaches, and can in principle justify its decisions in natural language. However, existing systems in this category typically condition the LLM's reasoning only on recent price statistics or raw text, without any structured, learned signal describing the broader statistical regime the market is currently in.

1.3.2 The Core Gap

Across all three families, no system combines (i) an explicit, learned representation of market regime, (ii) multi-perspective reasoning informed by that representation, and (iii) an execution layer that adapts its risk behavior according to regime and reasoning confidence. Regime is either ignored entirely, treated as a hidden variable an RL agent must discover implicitly, or left to ad hoc heuristics (e.g., a simple volatility threshold) rather than learned from data.

1.3.3 Formal Problem Statement

Given a multi-asset time series of market data, the problem addressed by this work is twofold:

1. **Regime representation learning**: learn a low-dimensional, temporally-aware embedding of market state that captures latent structure predictive of near-term dynamics, without relying on hand-crafted regime labels.

2. **Regime-conditioned decision making**: use this learned representation to condition both a multi-perspective reasoning process and a downstream execution policy, such that trading behavior adapts to the identified regime rather than applying a single fixed strategy across all market conditions.

Success is measured not only by trading performance (risk-adjusted returns, drawdown) but by whether regime conditioning demonstrably improves outcomes relative to regime-agnostic baselines — the central empirical question this report answers in Chapter 4.

1.4 Proposed Solution

MARTA addresses this gap through a five-layer pipeline, each layer directly targeting one part of the problem statement above.

1. **Feature Engineering**: a set of engineered technical and statistical indicators (volatility, momentum, trend, and higher-moment statistics) is computed per asset from hourly multi-asset crypto data (BTC, ETH, BNB, LTC, XRP), providing the market-state representation regime detection operates on directly, without requiring hand-labeled regimes. Two aspects of this layer's initial design were later revised (Chapter 3), both driven by the same finding: the representation was originally learned via a Temporal Fusion Transformer trained with an InfoNCE contrastive objective rather than engineered directly, and the initial data panel paired two crypto assets with two traditional-finance instruments (BTC, ETH, EUR/USD, XAU/USD) rather than the crypto-only panel above — a raw-feature HMM fit on

that mixed panel was found to collapse onto asset identity rather than market regime, motivating both revisions together.

2. **Regime Detection**: a Gaussian Hidden Markov Model is fitted directly on the engineered feature space, discretizing market state into a small number of interpretable, statistically distinct regimes with associated transition dynamics.

3. **Multi-Agent Deliberation**: five specialized LLM agents debate the appropriate trading stance given the current regime, each contributing a different analytical perspective (e.g., trend, risk, contrarian), producing a reasoned, regime-aware recommendation rather than a black-box signal.

4. **RL Execution Supervisor**: a PPO-trained agent translates the agents' deliberation output into concrete position sizing and execution decisions, learning to modulate risk exposure according to regime and the confidence of the upstream reasoning.

5. **Monitoring Dashboard**: a Next.js interface exposes regime detection, agent reasoning, and execution decisions in real time, making the system's behavior auditable rather than opaque.

This layered design directly mirrors the two-part problem statement: layers 1–2 solve the regime representation learning problem, and layers 3–4 solve the regime-conditioned decision making problem. Layer 5 is a supporting component providing interpretability rather than a decision-making stage in its own right. The empirical contribution of regime-awareness across layers 1–4 is evaluated through dedicated ablation studies, presented in Chapter 4.

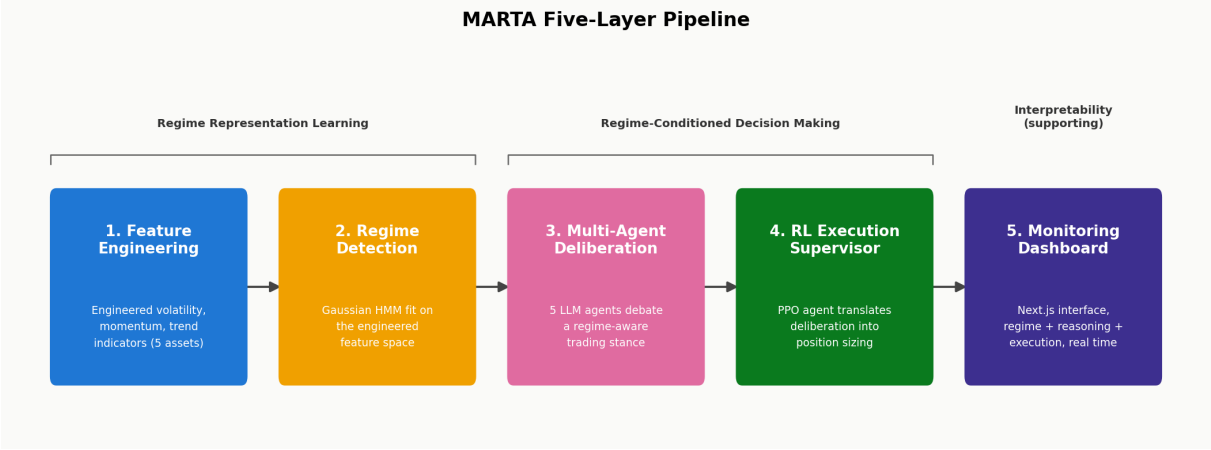


Figure 1.1: MARTA's five-layer pipeline, grouped according to the two-part problem statement of Section 1.3.3. Each layer is detailed in Chapter 3.

1.5 Project Objectives and Scope

1.5.1 Objectives

The project pursues four concrete objectives, each corresponding to a layer of the proposed solution:

1. **Learn a meaningful regime representation**: engineer a multi-asset hourly feature set (volatility, momentum, trend, and higher-moment statistics) that captures structure relevant to near-term market dynamics, validated through downstream regime separability once clustered by the HMM.

2. **Discover interpretable, statistically distinct regimes**: fit a Gaussian HMM directly over the engineered feature space and verify that the resulting regimes are stable, distinguishable, and align with intuitive market conditions (e.g., trending vs. ranging, high vs. low volatility) upon inspection.

3. **Build a regime-aware multi-agent reasoning layer**: design and implement a five-agent LLM deliberation system whose recommendations are conditioned on detected regime, and evaluate whether this conditioning improves recommendation quality over a regime-agnostic baseline.

4. **Train an adaptive execution policy**: implement a PPO-based execution supervisor that translates agent deliberation into position-sizing decisions, and evaluate — via ablation — whether regime-awareness measurably improves risk-adjusted trading performance relative to an equivalent regime-agnostic system.

A secondary, supporting objective is to expose the full pipeline through a monitoring dashboard, so that regime detection, agent reasoning, and execution decisions remain auditable rather than opaque — a property largely absent from prior work in this space (Section 1.3.1).

1.5.2 Scope

In scope:

- Five crypto assets: BTC/USDT, ETH/USDT, BNB/USDT, LTC/USDT, XRP/USDT, at hourly resolution, 2018–2025. An initial four-asset panel spanning crypto and traditional-finance instruments (BTC/USD, ETH/USD, EUR/USD, XAU/USD) was revised to this crypto-only panel after a data-quality defect specific to the traditional-finance instruments was diagnosed (Chapter 3).

- Design, training, and evaluation of all five architectural layers described in Section 1.4.

- Backtesting and ablation studies quantifying the contribution of regime-awareness specifically.

- A functional monitoring dashboard for the full pipeline.

Out of scope:

- Live deployment with real capital; the system is evaluated via backtesting and paper-trading simulation only.

- Assets or timeframes beyond the five instruments and hourly resolution listed above.

- Regulatory, compliance, or brokerage-integration work required for production trading.

- Optimization of LLM inference cost/latency for high-frequency use cases; the multi-agent layer operates at hourly decision frequency, not tick-level.

This scope reflects the project's nature as an applied research contribution demonstrating the value of regime-awareness, rather than a production-ready trading product.

1.6 Requirements Specification

The objectives of Section 1.5 state *what* the project set out to demonstrate; this section restates the same commitments as concrete, individually checkable requirements, in the functional/non-functional form expected of an engineering deliverable, and points each one to the chapter and section where it is actually discharged and evidenced, rather than only asserted here.

1.6.1 Functional Requirements

- **FR1 — Regime representation.** The system shall derive a market-state representation from multi-asset OHLCV data without relying on hand-labeled regimes. *Discharged by:* Section 3.3 (feature engineering) and Section 3.4 (the superseded learned-encoder variant).
- **FR2 — Regime detection.** The system shall discretize market state into a small number of statistically distinct regimes with explicit transition dynamics, and that distinctness shall be verified, not assumed. *Discharged by:* Section 3.5; verified in Section 4.1.1 (volatility distinctness, $p \approx 0$ across every configuration tested).
- **FR3 — Regime-conditioned multi-agent reasoning.** The system shall produce a directional trading recommendation from multiple specialized perspectives, each explicitly conditioned on the detected regime rather than reasoning over price alone. *Discharged by:* Section 3.6; verified in Section 4.2.1 (the null-regime test, confirming regime context is not silently discarded).
- **FR4 — Learned execution policy.** The system shall translate agent consensus and portfolio state into position-sizing decisions via a trained policy rather than a fixed rule. *Discharged by:* Section 3.7.
- **FR5 — Live monitoring.** The system shall expose regime detection, agent reasoning, and execution decisions through a real-time interface. *Discharged by:* Section 3.9.
- **FR6 — Decision auditability.** Every displayed decision (action, position size, conviction) shall be traceable to the specific inputs — regime, consensus, risk limits — that produced it, and every quantity shown alongside a decision shall originate from the same evaluation run as that decision. *Discharged by:* Section 3.9.7.

1.6.2 Non-Functional Requirements

- **NFR1 — Evaluation determinism.** Repeated evaluation of an identical checkpoint under an identical configuration shall return bit-identical metrics, so that a reported result is attributable to the checkpoint and configuration rather than to run-to-run noise. *Discharged by:* Section 3.7.5.

- **NFR2 — Data integrity / no leakage.** No training, model-selection, or hyperparameter decision shall read data from the test split; this boundary shall be asserted in code, not only documented. *Discharged by:* Section 3.3.3.
- **NFR3 — Statistical rigor.** Every headline performance comparison shall be reported with a significance test or an explicit noise control, not as an isolated point estimate. *Discharged by:* Section 3.8.3; applied throughout Chapter 4.
- **NFR4 — Modularity and maintainability.** The five architectural layers shall be independently revisable — a change to one layer (e.g. the regime detector's asset panel) shall not require redesigning the layers downstream of it. *Discharged by:* the layered design of Section 1.4; exercised in practice when the asset-panel revision of Section 3.5.6 and the encoder's retirement (Section 3.4.4) were both absorbed without a redesign of the multi-agent or dashboard layers.
- **NFR5 — Operational responsiveness.** The live system shall produce one regime/consensus/execution decision per asset per hour, matching the pipeline's native data resolution. *Discharged by:* Section 3.9.2 (the APScheduler-driven hourly signal tick).

1.7 Project Methodology and Work Planning

Given the exploratory, research-oriented nature of MARTA, a rigid upfront specification was not appropriate: each architectural layer's design depended on empirical results from the layer built before it (e.g., the number and structure of latent regimes discovered by the HMM directly shaped how agent prompts were engineered). The project therefore followed an **iterative, layer-by-layer methodology**, where each phase produced a working, evaluable component before the next was started, rather than a big-bang integration at the end.

1.7.1 Choice of Methodology: CRISP-DM over Sprint-Based Agile

Sprint-based Agile frameworks (Scrum, Kanban) were considered and set aside. Both assume a backlog of comparatively independent, estimable units of work that can be committed to a fixed-length iteration; MARTA's work plan did not have that shape. The content of a given phase's deliverable — how many regimes the HMM would discover, whether the learned encoder would outperform raw features, which architectural lever (if any) would close the RL layer's gap to buy-and-hold — was not knowable until the previous phase's empirical results were in hand, so committing a sprint's scope in advance would have meant committing to work that later evidence twice required reopening (Section 3.5.6's panel revision and Section 3.4.4's encoder retirement, both triggered by evaluation outcomes rather than schedule). CRISP-DM was adopted instead specifically because its Evaluation phase is a first-class, named step that can route back to Data Preparation or Modeling, giving this evidence-driven revision pattern an explicit place in the methodology rather than treating every such revision as an exception to a Scrum backlog planned before the evidence existed.

1.7.2 Development Methodology: CRISP-DM

MARTA's process follows CRISP-DM (Cross-Industry Standard Process for Data Mining) [1], whose six canonical phases — Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment.

CRISP-DM's defining property is that it is not a one-way pipeline: it explicitly allows evaluation in one phase to send the project back to an earlier one, and MARTA's own revision history is a direct instance of this rather than a deviation from a plan. Three examples recur throughout Chapter 3: evaluating the regime encoder against a raw-feature baseline (Evaluation) found it underperforming and sent the design back to Modeling, where the HMM was refit directly on engineered features instead (Section 3.4.4); evaluating the resulting model against the original four-asset panel (Evaluation) found it had collapsed onto asset identity rather than market regime, sending the project back to Data Preparation to redefine the asset panel itself (Section 3.5.6); and evaluating the RL execution layer's ablation (Evaluation) found no configuration beat buy-and-hold, changing the Deployment phase's default signal source accordingly (Section 4.3.3). Table 1.2's nine work-plan phases are this same six-phase cycle instantiated concretely for MARTA, several of them spanning more than one CRISP-DM phase where a revision folded two stages back together.

Table 1.1: CRISP-DM phases

CRISP-DM Phase	**Report Section(s)**
Business Understanding	Section 1.3 (Problem Statement), Section 1.4 (Proposed Solution), Section 1.5 (Objectives and Scope), Section 1.6 (Requirements Specification)
Data Understanding	Section 3.3.1 (Data Sources and Coverage); Section 3.5.6 (Asset Panel Revision)
Data Preparation	Section 3.3.2 (Pipeline Stages), Section 3.3.3 (Train/Validation/Test Split)
Modeling	Section 3.4 (Regime Encoder), Section 3.5 (Regime Detection), Section 3.6 (Multi-Agent Deliberation), Section 3.7 (RL Execution Agent)
Evaluation	Section 3.8 (Evaluation Methodology), Chapter 4 (Results)
Deployment	Section 3.9 (Dashboard and Live System)

This mapping tracks Chapter 3's own section order, which follows the same Data → Modeling → Evaluation → Deployment sequence without having been deliberately arranged to do so.

1.7.3 Development Phases

The work was organized into nine phases, summarized in Table 1.2 and Figure 1.2. The first five ran sequentially: literature review and architecture design, then one phase per architectural layer through the multi-agent system, each producing a working, evaluable component before the next began. The final four phases (RL execution, dashboard/live integration, encoder diagnostics, and the raw-feature pivot) overlapped rather than running strictly in sequence — a genuine reflection of the project's last two months, during which RL training, dashboard development, and a systematic audit of the encoder were

pursued concurrently, converging in the decision to drop the learned encoder in favor of engineered features (Chapter 3).

Table 1.2: MARTA development phases

Phase	**Deliverable**	**Duration & dates**	**CRISP-DM Phase(s)**
1. Literature Review & Architecture Design	State-of-the-art survey, 5-layer design	8 wk (Mar 16 – May 10)	Business Understanding
2. Data Pipeline & Feature Engineering	Cleaned multi-asset hourly dataset (2018–2025)	2 wk (May 11 – May 24)	Data Preparation
3. Regime Encoder (TFT + InfoNCE) — v1	Trained TFT + InfoNCE encoder (later superseded, Ch. 3)	3 wk (May 25 – Jun 14)	Modeling
4. Regime Detection (HMM)	Gaussian HMM over encoder embeddings (later refit on raw features, Ch. 3)	2 wk (Jun 15 – Jun 28)	Modeling
5. Multi-Agent System	5-agent LLM debate pipeline	3 wk (Jun 29 – Jul 19)	Modeling
6. RL Execution Agent & Evaluation Fixes	PPO supervisor trained on agent outputs; eval-determinism and reward-scaling fixes	6 wk (Jul 20 – Aug 30)	Modeling, Evaluation
7. Dashboard & Live Integration	Next.js dashboard, live data bridges, paper-trading execution	6 wk (Jul 27 – Sep 6)	Deployment
8. Encoder Diagnostic & Rejected Retrains	Systematic audit of encoder retrain variants; none promoted (Ch. 3)	3 wk (Aug 10 – Aug 30)	Evaluation
9. Raw-Feature Pivot & Crypto5 Migration	Encoder retired in favor of engineered features; crypto-only 5-asset panel adopted	3 wk (Aug 27 – Sep 16)	Data Understanding, Data Preparation, Modeling

1.7.4 Rationale for the Approach

This phase-by-phase structure offered two advantages over a monolithic design-then-build approach. First, it allowed early validation of the encoder and regime detection stages in isolation — verifying, for instance, that discovered regimes were interpretable and stable before committing to an agent architecture built on top of them. Second, it kept technical risk contained: had the HMM failed to produce meaningful regime clusters, the multi-agent and RL layers could have been redesigned without discarding the data pipeline or encoder work. The overlap in phases 6–9 is itself part of that story: it was the concurrent pressure of RL evaluation work and a systematic encoder audit — not a planned step — that surfaced the case for the raw-feature pivot.

The sections of Chapter 3 broadly follow this same progression; the phase 8–9 findings are folded into the discussion of the layers they ultimately affected (the Regime Encoder

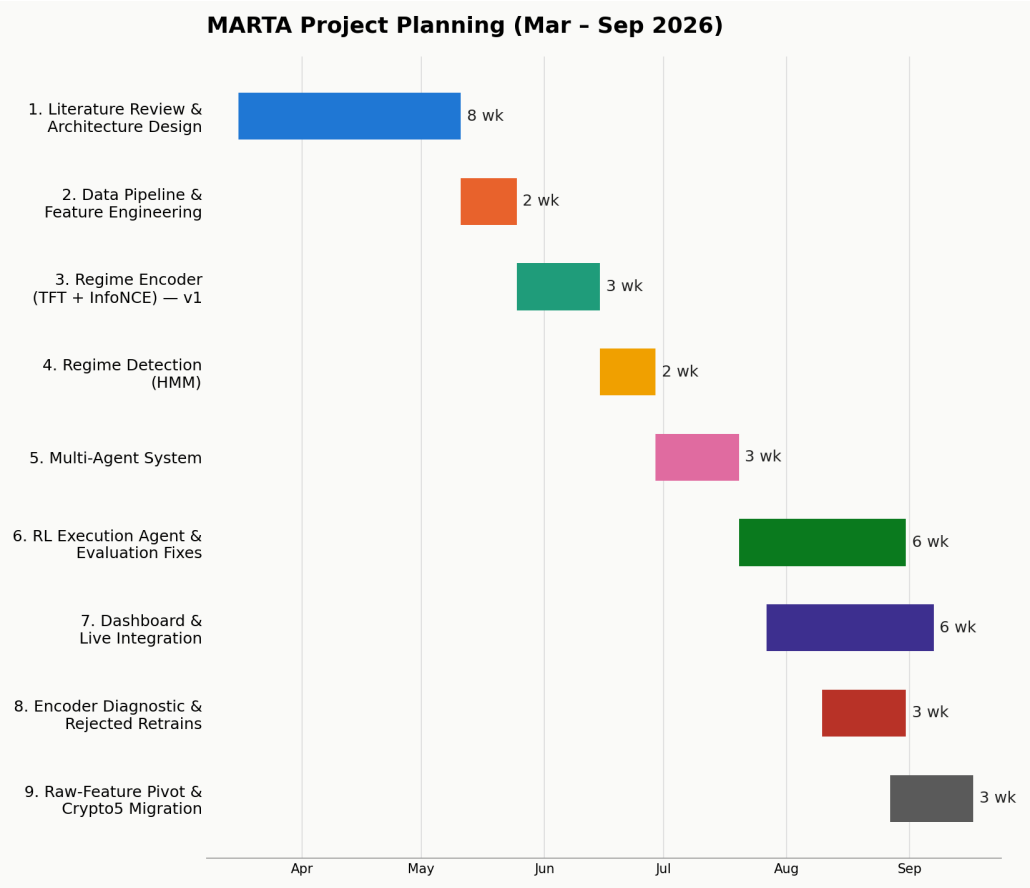


Figure 1.2: Project planning Gantt chart

and Regime Detection sections) rather than given standalone sections of their own.

Chapter 2

State of the Art

2.1 Introduction

This chapter surveys the three bodies of work MARTA draws on and positions itself against: market regime detection, LLM-based trading agents, and reinforcement learning for trading. Section 2.2 briefly traces how algorithmic trading arrived at its current state. Sections 2.3–2.5 review each of the three families in turn, closing each with the specific limitation that motivates one part of MARTA's design. Section 2.6 then consolidates these gaps into a single comparative view and states precisely where MARTA sits relative to prior work.

2.2 Algorithmic Trading: Evolution and Challenges

Automated trading systems have progressed through three broad generations. The first generation consisted of rule-based strategies — moving-average crossovers, momentum filters, mean-reversion bands — valued for their transparency but structurally unable to adapt once market conditions diverge from the assumptions baked into their fixed parameters. The second generation applied supervised and reinforcement learning to trading directly, allowing strategies to be learned from data rather than hand-specified, at the cost of interpretability. The third and most recent generation incorporates large language models, exploiting their capacity to reason over unstructured text such as news and analyst commentary rather than numerical features alone. Each generation solved a limitation of the one before it while introducing a new one, a pattern examined in detail across the following three sections.

2.3 Market Regime Detection

2.3.1 Rule-Based Approaches

The simplest regime-detection methods rely on hand-crafted thresholds over indicators such as realized volatility or moving-average slope to classify the market into discrete states (e.g., trending vs. ranging). These approaches are computationally trivial and easy to interpret, but the thresholds themselves must be manually tuned per asset and require periodic re-calibration as market conditions evolve, making them brittle outside the conditions they were tuned on.

2.3.2 Statistical Approaches: Hidden Markov Models and Clustering

Hidden Markov Models (HMMs) have long served as the primary statistical tool for regime detection in finance, following the foundational regime-switching framework introduced by Hamilton for modeling business-cycle turning points in macroeconomic time series [2]. Applied to financial returns, an HMM treats the market regime as an unobserved latent state that generates observable return and volatility patterns, with transition probabilities between states estimated directly from data rather than imposed by rule. Gaussian HMMs fitted over hand-engineered feature sets (returns, realized volatility) have been shown to recover interpretable bull/bear/consolidation regimes on major equity indices, and have been used as risk-managing filters that suppress trading during predicted high-volatility states. K-means and Gaussian Mixture clustering offer a simpler, non-sequential alternative, but typically yield only two well-separated clusters (bullish/bearish) and lack the transition-probability structure that makes HMMs useful for characterizing regime persistence and switching dynamics.

The central limitation of this family, for MARTA's purposes, is the feature representation the HMM operates on: virtually all published work fits the HMM directly over hand-selected statistical features (returns, volatility, sometimes volume), rather than over a representation learned to capture broader temporal structure. This caps the ceiling of what the HMM can discover — it can only find regimes expressible in terms of the features it was given.

2.3.3 Deep Learning Approaches

More recent work combines regime detection with deep reinforcement learning rather than treating it as a standalone preprocessing step. A 2025 study integrating an HMM-based regime filter directly into the FinRL reinforcement learning framework found that regime-aware hybrid agents outperformed standard regime-agnostic FinRL baselines on risk-adjusted return metrics across a 30-stock portfolio, providing direct empirical support for the core hypothesis underlying MARTA: that explicit regime conditioning improves trading outcomes relative to regime-agnostic learning. However, this and comparable work still fits the HMM over hand-engineered statistical features rather than a learned temporal representation, and pairs regime detection only with an RL execution layer — it does not incorporate any reasoning layer capable of explaining *why* a regime implies a particular stance, which is precisely the gap MARTA's multi-agent layer is designed to fill.

2.4 LLM Agents in Finance

2.4.1 Single- and Multi-Agent LLM Trading Systems

TradingGPT [3], an early multi-agent system, assigns LLM agents layered memory and distinct behavioral characters to simulate diverse trader personas contributing to a shared decision. FinAgent, presented at KDD 2024, is a multimodal foundation agent that fuses textual news, numerical price data, and visual chart information through a dual-level reflection module, allowing the agent to adapt its strategy based on recent market feedback; evaluated across six financial datasets spanning equities and crypto, it substantially out-

performed prior single-modality baselines. TradingAgents, proposed shortly after, more directly mirrors the organizational structure of a real trading firm: specialized analyst agents (fundamental, sentiment, technical) feed a research team of Bull and Bear debaters, whose output passes through a risk management team before a trader agent commits to a decision, and the resulting framework outperformed several classical baselines (Buy-and-Hold, MACD, RSI) on cumulative return and Sharpe ratio in simulated equity trading.

2.4.2 Multi-Agent Debate as a Design Pattern

The common thread across TradingGPT, FinAgent, and TradingAgents is the use of multiple specialized agents whose outputs are synthesized rather than a single monolithic LLM call — a design pattern motivated by the observation that decomposing analysis into distinct perspectives (technical, sentiment, risk) produces more robust recommendations than asking one model to reason about all of them simultaneously. MARTA's five-agent deliberation layer adopts this same debate pattern.

2.4.3 Limitations of Existing Approaches

None of the systems reviewed above condition their agents' reasoning on a learned, structured representation of market regime. TradingAgents' agents reason over recent price statistics and text directly; FinAgent's reflection module adapts based on recent performance feedback rather than an explicit regime signal; TradingGPT's characters are fixed personas rather than regime-conditioned roles. In each case, if the notion of "market regime" appears at all, it is implicit in whatever the LLM infers from raw inputs at inference time, not an explicit, pretrained signal shared across agents. This is the specific gap MARTA's regime encoder and HMM layer are designed to fill upstream of the agent debate.

2.5 Reinforcement Learning for Trading

2.5.1 Early Approaches: DQN and Discrete Action Spaces

Early applications of deep RL to trading adapted Deep Q-Networks (DQN) to discrete action spaces (buy/sell/hold), following the broader deep RL literature. FinRL, an open-source framework from Columbia University and NYU Shanghai, standardized this line of work by providing a full pipeline — market environments, fine-tuned implementations of DQN, DDPG, PPO, SAC, A2C, and TD3, and common reward functions — specifically to lower the engineering barrier for applying deep RL to quantitative trading. DQN-based agents are conceptually simple but constrained to discrete position sizes, limiting their ability to express graduated risk exposure.

2.5.2 PPO and Continuous Action Spaces

Proximal Policy Optimization (PPO), introduced by Schulman et al., addresses the instability of earlier policy gradient methods through a clipped surrogate objective that constrains the size of each policy update, achieving much of the stability of Trust Region Policy Optimization while being substantially simpler to implement. Its compatibility

with continuous action spaces makes it well suited to position-sizing tasks, where the agent must output a continuous fraction of capital to allocate rather than a discrete action, and it is the algorithm implemented for MARTA's execution supervisor for exactly this reason.

2.5.3 Curriculum Learning in Finance

Curriculum learning — structuring training so an agent first masters a simpler version of a task before progressing to a harder one — is comparatively underused in financial RL relative to other RL application domains, though the general principle (start with a single, liquid, well-behaved asset before extending to a more volatile or lower-liquidity one) aligns with standard practice in curriculum design more broadly. MARTA's own BTC-to-BTC+ETH curriculum training, detailed in Chapter 3, follows this logic.

2.6 Positioning of MARTA

Table 2.1 consolidates the gaps identified across the three families reviewed above.

Table 2.1: Comparison of MARTA against representative prior systems

System	**Learned regime repr.**	**Multi-agent reasoning**	**Adaptive execution**	**Interpretable**
HMM regime filters [4]	Partial (hand-features)	No	No	Yes
FinRL [5]	No	No	Yes (RL)	Partial
HMM + FinRL [6]	Partial (hand-features)	No	Yes (RL)	Partial
TradingGPT [3]	No	Yes	No	Partial
FinAgent [7]	No	Partial (single agent)	No	Yes
TradingAgents [8]	No	Yes	No	Yes
MARTA (this work)	**Yes (Gaussian HMM, engineered feats.)**	**Yes (5 agents)**	**Yes (PPO)**	**Yes**

No system reviewed combines a learned regime representation, multi-agent reasoning conditioned on that representation, and an adaptive execution layer in a single pipeline. This is precisely the combination MARTA proposes: a Gaussian Hidden Markov Model fit directly on engineered market features learns the regime representation, producing an explicit probabilistic regime posterior rather than the hand-tuned volatility thresholds that ad hoc regime filters rely on; this representation conditions the multi-agent debate that LLM trading systems otherwise perform without any explicit regime signal; and a PPO-based execution supervisor [9] translates the resulting recommendation into position sizing, closing the loop that regime-detection-only systems leave open.

2.7 Summary

This chapter reviewed market regime detection, LLM-based trading agents, and reinforcement learning for trading as three largely separate research threads, each addressing part of the problem MARTA targets while leaving a specific gap: regime detection methods rely on hand-crafted features rather than learned representations; LLM agent systems reason without any explicit regime conditioning; and RL execution methods, even when regime-aware, lack an interpretable reasoning layer. Chapter 3 now turns to how MARTA's five-layer architecture, introduced in Section 1.4, is designed and implemented to close these gaps jointly.

Chapter 3

Methodology

3.1 Overview

This chapter details the design and implementation of MARTA's five-layer architecture, introduced in Section 1.4 and developed across Phases 2–7 of the project plan (Table 1.2). Each section below corresponds to one development phase, in the same order the system was built: Section 3.3 covers the data pipeline (Phase 2), Section 3.4 the contrastive regime encoder (Phase 3), Section 3.5 the Gaussian HMM regime detector (Phase 4), Section 3.6 the five-agent LLM deliberation system (Phase 5), Section 3.7 the PPO execution agent (Phase 6), and Section 3.9 the monitoring dashboard and live-trading bridge (Phase 7). Section 3.8 describes the evaluation protocol — walk-forward splitting, the ablation design, and the statistical testing procedure — used to produce the results reported in Chapter 4.

As established in Section 1.7, each layer's design was constrained by empirical results from the layer built before it; where a design decision was revised after an earlier choice proved inadequate, both the original approach and the reason for revising it are reported, since the revision itself is part of the project's methodology.

3.2 System Design and UML Models

Before detailing each layer's internal design, this section fixes four complementary views of the system as a whole — who interacts with it, how its core classes relate, how a single decision cycle actually flows at runtime, and how it is deployed — using the standard UML diagram types for each. Figure 1.1 (Section 1.4) already gives the data-flow view between the five layers; the models below do not repeat it, and instead cover structural and behavioral aspects that figure does not.

3.2.1 Use Case Model

Figure 3.1 identifies two human actors and three external systems the platform interacts with. The **ML/Research Engineer** role covers the data and modeling use cases that precede a live decision — ingesting and normalizing OHLCV data (UC1), engineering indicators (UC2), training or redeploying the regime detector, the multi-agent system, and the RL policy (UC3, UC6, UC14), running backtests and ablations (UC12), and auditing the decision log (UC11). The **Trader/Portfolio Manager** role covers operational use cases layered on top of the already-running system — querying it in natural language

(UC10), toggling RL execution on or off (UC13), and receiving alerts (UC9). Several use cases `<<include>>` a lower-level one by construction rather than by convention: running the multi-agent debate (UC4) always includes detecting the current regime (UC3), and training or serving the RL policy (UC6) always includes computing risk limits (UC5), since both are called synchronously on the critical path of a single decision rather than as separate, independently triggerable operations. The three external systems — Ollama (local LLM inference), the Binance/CCXT market-data feed, and the Alpaca paper-trading API — are modeled as actors specifically because the system has no control over their availability or response time, a property that directly motivated the timeout and retry handling described in Section 3.6.4.

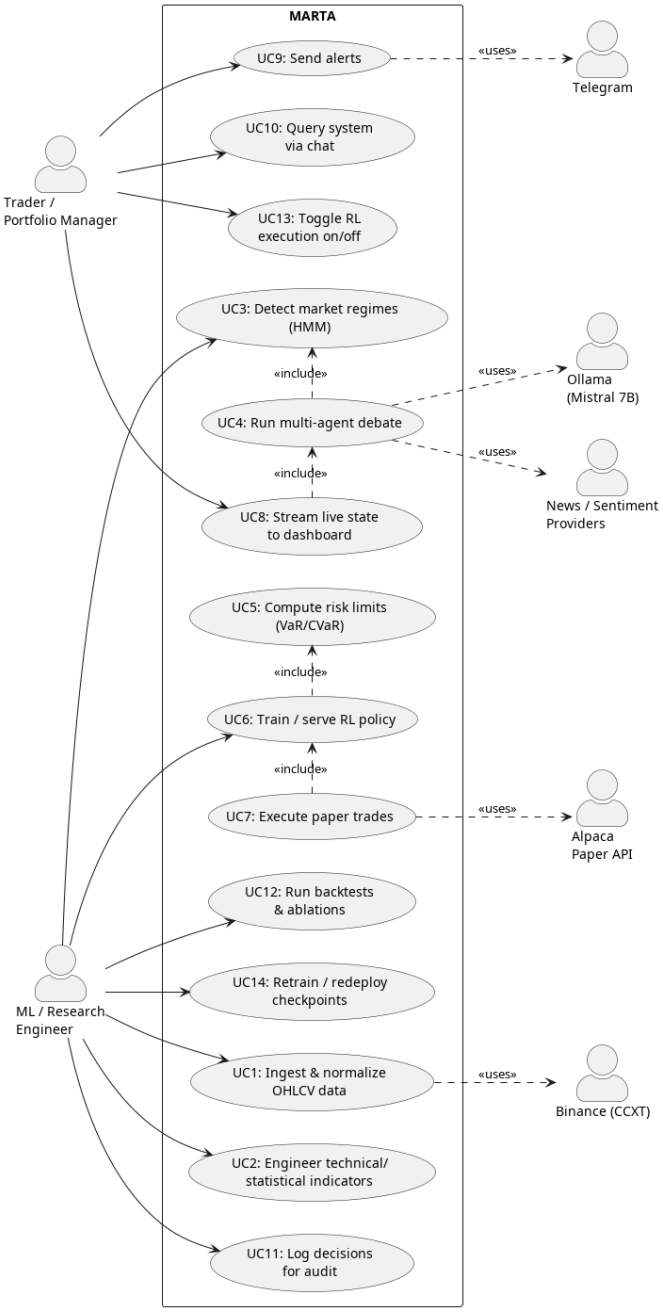


Figure 3.1: Use case diagram: actors, use cases, and `<<include>>`/`<<uses>>` relationships across the five layers.

3.2.2 Class Model

Figure 3.2 shows the core classes behind the multi-agent and RL layers detailed in Sections 3.6 and 3.7. All five directional and risk agents (`MacroAgent`, `TechnicalAgent`, `SentimentAgent`, `ContrarianAgent`, `RiskAgent`) inherit from a common `BaseAgent`, which owns the LLM call, the five-stage JSON parse-recovery logic, and the timeout guard described in Section 3.6.4 — centralizing that logic in one base class rather than duplicating it per agent is what let the five-stage recovery fix apply uniformly everywhere at once. `FundManagerAgent` is deliberately not a subclass of the same hierarchy: it consumes a `DebateResult` (itself composed of `AgentThesis` and `AgentRebuttal` instances) and a `RiskLimits` object to produce a `FundManagerDecision`, reflecting that it synthesizes other agents' output rather than independently forming a directional view. `RegimeContext` is the object threaded through every agent call, produced by the `RegimeHMM` and consumed by every `BaseAgent` subclass, which is how a single regime fit is shared across an entire debate round without being refit per agent. On the execution side, `MARTATradingEnv` (the Gymnasium environment of Section 3.7.2) produces the `EnvState` consumed by `MARTAActorCriticPolicy`, keeping the environment and the policy's network architecture as separate classes connected only through that typed state object.

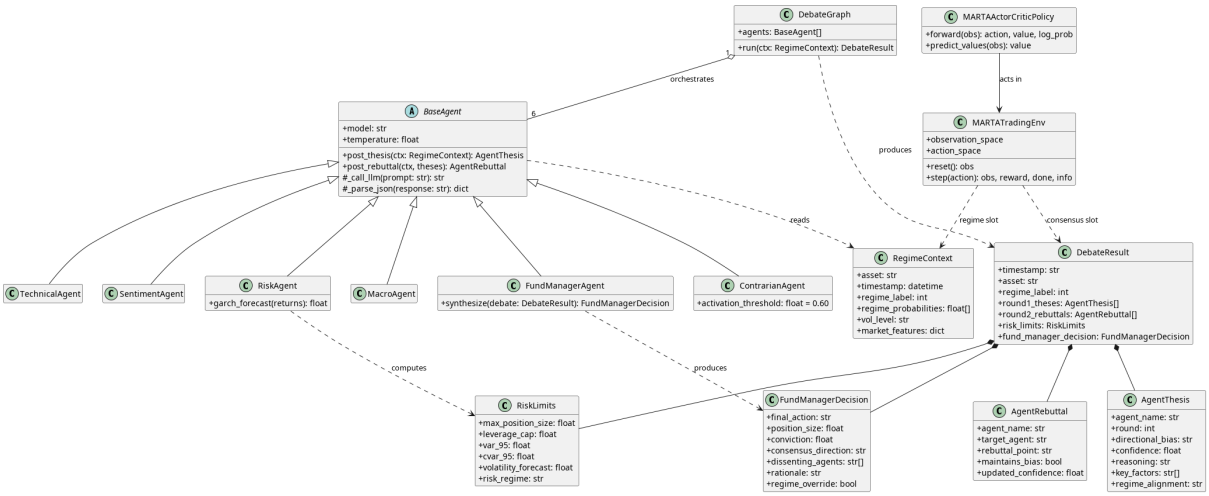


Figure 3.2: Class diagram: agent hierarchy, debate data classes, and RL environment/policy classes.

3.2.3 Sequence Model

Figure 3.3 traces one full decision cycle end to end, from a new hourly OHLCV bar arriving from Binance to a paper order being placed and a Telegram alert being sent — the same cycle described narratively in Section 3.9.2, shown here as the concrete sequence of calls across every object in Figure 3.2. The data pipeline (steps 1–4) runs the `features.py/normalize.py` stages of Section 3.3.2 synchronously on the new bar before anything downstream executes. The Gaussian HMM's `predict_proba()` call (step 6) produces the `RegimeContext` that both the debate (steps 8–18) and, as a separate state slot, the RL policy (step 19) consume — the diagram makes explicit that these are two independent consumers of the same regime fit, not a chain where the debate's output determines the regime seen by the policy. Round 2 rebuttals (step 14) and the

Contrarian agent (step 13) are shown as conditionally executed, matching the activation rules of Section 3.6.4 rather than running unconditionally on every cycle. The final two steps (23–24) show the `FundManagerDecision`/RL action reaching `MARTATradingEnv` and, from there, both the Alpaca paper-execution and Telegram-alert side effects firing from the same resolved trade signal.

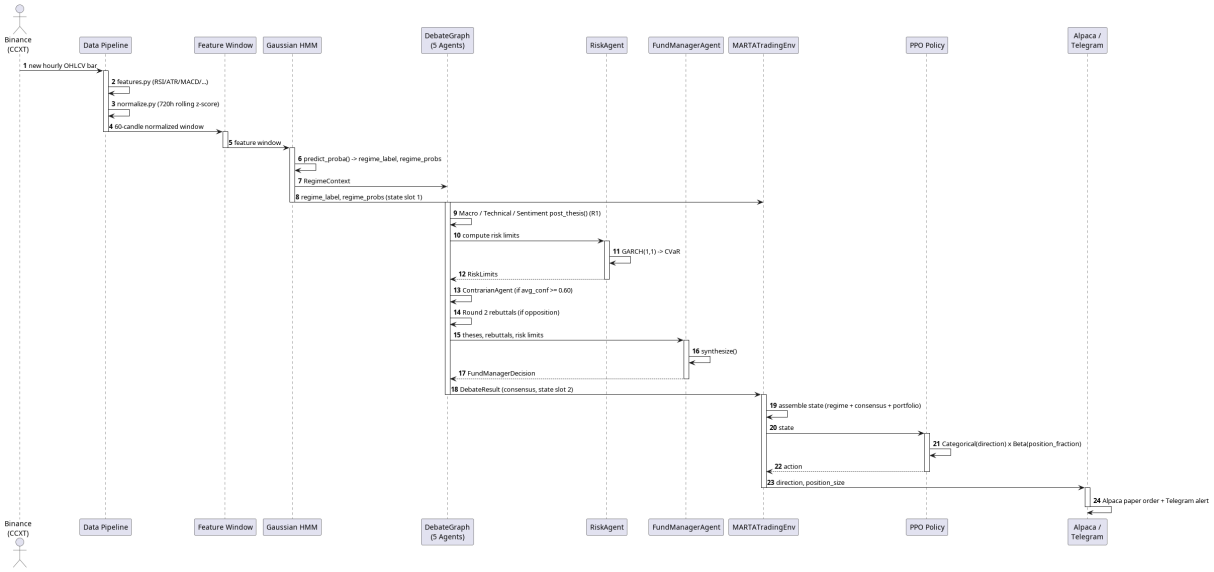


Figure 3.3: Sequence diagram: one hourly decision cycle, from a new OHLCV bar to a placed paper order and dispatched alert.

3.2.4 Deployment Model

Figure 3.4 shows the system as actually run, matching the Docker-based deployment described in Section 3.9.2: a frontend container (Next.js) and a backend container (FastAPI/Uvicorn) on one Docker host, communicating over REST and WebSocket on port 8001. Inside the backend container, the APScheduler hourly tick and the REST/WebSocket API are shown as separate components sharing the same process, since the scheduler calls directly into the same service modules the API routes call rather than communicating through a queue — consistent with the project's deliberate choice of APScheduler over Celery/Redis (Section 3.9.2). Ollama runs as a host process rather than inside a container, reached over `host.docker.internal`, because it needs direct GPU access that the containerized services do not. The `paper_trading_db` SQLite volume and the `data//mlruns/` mounted volumes are shown as distinct from each other because they have different lifecycles: the former is written continuously by live paper trading, while the latter is written by offline training/evaluation runs and read, not written, by the backend container. Five external services — Binance/CCXT, Twelve Data, the Alpaca Paper API, the Telegram Bot API, and the news/sentiment feeds — are reached over HTTPS from the backend only, never from the frontend container directly.

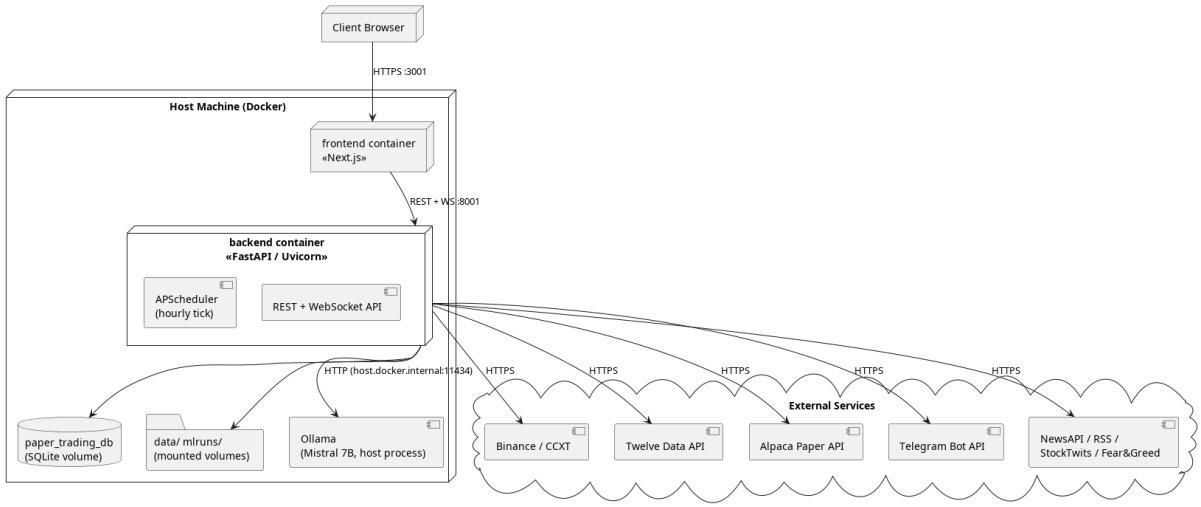


Figure 3.4: Deployment diagram: Docker containers, mounted volumes, the host-process LLM, and external service dependencies.

3.3 Data and Feature Engineering

3.3.1 Data Sources and Coverage

This subsection describes MARTA's initial data panel and pipeline, mixing crypto and traditional-finance instruments. Section 3.5.6 documents why this panel was later revised to a crypto-only, five-asset universe (BTC, ETH, BNB, LTC, XRP) — the panel actually used to produce the regime, multi-agent, and RL results reported in Chapter 4 — and gives that panel's own data-sourcing details. The pipeline stages, feature set, and train/validation/test split described below (Sections 3.3.2–3.3.3) are unaffected by the panel revision and apply identically to both panels.

MARTA's initial data panel operated on four hourly instruments spanning two asset classes: **BTC/USDT** and **ETH/USDT**, fetched via the CCXT library against Binance, and **EUR/USD** and **XAU/USD** (gold), fetched from histdata.com's genuine 1-minute tick archives and resampled to hourly OHLCV (`open`=first, `high`=max, `low`=min, `close`=last, `volume`=sum). A fifth instrument, WTI crude oil, was dropped from the pipeline entirely: its exchange-holiday and weekend gaps were the binding constraint on cross-asset timestamp alignment, and removing it substantially improved the overlap between the crypto (24/7) and traditional-asset (weekday-only) series without any loss of coverage on the four remaining assets.

The crypto series are pulled forward continuously and reach mid-2026 at the time of writing; the forex/commodity series are capped at 2025-12-31 because histdata.com only publishes complete-year archives and no 2026 archive yet exists. This asymmetry is immaterial to the encoder, HMM, and RL training splits below (all of which end well before 2025), but is directly relevant to the live-trading bridge described in Section 3.9.

An earlier version of the EUR/USD and XAU/USD pipeline used daily yfinance data forward-filled to hourly resolution, which produced 23 near-duplicate hourly bars for every genuine daily observation and measurably contaminated the contrastive embedding space. Switching to histdata.com's genuine 1-minute data, resampled properly, removed this artifact.

3.3.2 Pipeline Stages

The pipeline runs as five sequential stages (`fetch` → `features` → `normalize` → `align` → `validate`), each operating on one asset's data independently until the alignment stage joins them:

1. **Fetch**: retrieves raw OHLCV via CCXT (crypto, batched request loop) or the histdata year-ZIP archive (forex/commodities), caching raw and intermediate artifacts to disk so repeated runs only fetch the missing tail. A sanitization pass enforces `high` = max(open,high,close) and `low` = min(open,low,close), and forward-fills zero-valued closes, guarding against malformed exchange data reaching any downstream stage.

2. **Features**: computes 13 technical indicators from the 5 raw OHLCV columns using the `ta` library and hand-rolled statistics, producing an 18-column intermediate frame (Table 3.1).

3. **Normalize**: applies a rolling 720-hour (30-day) z-score to 12 of the 13 indicators (minimum 168 hours of history required before a window is emitted), drops the raw OHLCV columns, and retains `close_return` unmodified as the return target. One indicator, `realized_vol_raw`, is deliberately excluded from normalization (see below).

4. **Align**: inner-joins all four assets' normalized frames on the BTC timestamp index (BTC/USDT trades 24/7 and is therefore the backbone), producing one combined parquet with MultiIndex columns `(asset, feature)` alongside per-asset parquets.

5. **Validate**: runs structural checks (column presence, dtype, no unexpected NaNs — CRITICAL severity), value-range checks, gap and distribution checks (WARNING severity), OHLCV sanity checks, and cross-asset timestamp-overlap checks, logging every metric to MLflow and writing a JSON validation report.

FX/commodity data from histdata.com carries no centralized volume figure (unlike a single-exchange crypto feed), so `obv_raw` and `vol_zscore` are near-constant for EUR/USD and XAU/USD; for these two assets, the price-derived indicators carry the useful signal.

3.3.3 Train / Validation / Test Split

All downstream components — encoder pretraining, HMM fitting, and RL training — share a single, strictly walk-forward split, defined once in a canonical module and imported everywhere it is needed rather than re-specified per component:

Split	**Window**
Train	2018-01-01 $\leq$ t $<$ 2024-01-01
Validation	2024-01-01 $\leq$ t $<$ 2025-01-01
Test	2025-01-01 $\leq$ t $<$ 2026-01-01

The three splits are contiguous by construction (the train/validation boundary equals the validation/test boundary of the previous pair, with no gap and no overlap), and this invariant is asserted at import time rather than merely documented. The test split is treated as sacred: no code path involved in encoder training, HMM/PCA fitting, or RL

Table 3.1: The 14 features per asset used throughout the pipeline

Feature	Description
atr, adx, rsi, bb_width, macd_histogram	Standard technical indicators (Average True Range, Average Directional Index, Relative Strength Index, Bollinger Band width, MACD histogram), rolling z-scored.
obv_raw	On-Balance Volume, raw cumulative sum at feature-computation time; z-scored once (over 720h) at the normalize stage — an earlier version applied a second, redundant 168h z-score on top, which was removed.
vol_zscore, realized_vol, vol_ratio	Volatility-family indicators, rolling z-scored.
return_skew, return_kurtosis, autocorr_lag1	Higher-moment and autocorrelation statistics of returns, rolling z-scored.
realized_vol_raw	Unnormalized annualized realized volatility — the one feature the normalize stage does *not* z-score, so it preserves absolute magnitude. Included specifically because the z-scored realized_vol compresses extreme events (e.g. the March 2020 crash) into an ordinary-looking z-score, making them invisible to any downstream component that only sees the normalized value.
close_return	Simple hourly return, passed through unnormalized as the return target used by GARCH fitting and reward computation.

training or model selection is permitted to read a 2025 timestamp, and each of those three components independently re-verifies, after applying its own slice of the data, that the resulting slice does not cross the test boundary — catching a masking bug in the slicing logic itself, not just a mis-specified date constant. Test-set evaluation is performed exclusively through one dedicated, separate evaluation script, so that a mistake in an exploratory or diagnostic script can never silently contaminate model selection.

This three-way split, and the discipline around it, exists because every earlier iteration of the project used a shorter or differently-placed boundary (see Section 3.8); consolidating it into one canonical, self-validating module was a direct response to boundaries silently going stale in individual files during the several data-range extensions the project underwent.

3.4 Regime Encoder

3.4.1 Design Objective

This section describes MARTA's initial design for Layer 1's market-state representation, later superseded in production by fitting the regime detector directly on engineered features (Section 3.5.6). It is kept here because the encoder's own diagnosed shortcoming (Section 3.4.4) is what motivated that revision, and because it is the design actually built and evaluated in this phase of the project.

Layer 1 learns a fixed-length embedding of the 14-feature, 60-candle window ending at each hourly timestamp, without hand-labeled regimes, via a self-supervised contrastive objective (InfoNCE [10]): two augmented *views* of the same window are pulled together in

embedding space while views of unrelated windows are pushed apart — standard SimCLR-style pretraining, applied here to financial time series instead of images.

3.4.2 Architecture

The encoder is *TFT-inspired* rather than a full Temporal Fusion Transformer [11]: it keeps a TFT encoder path's core components — a Variable Selection Network (VSN), a recurrent sequence encoder, and a gated residual connection — but drops the multi-horizon decoder, static covariate encoders, and attention block, none needed for a fixed-window embedding task with no forecasting horizon. Table 3.2 lists the layers in forward-pass order.

The VSN's per-timestep weights are also reused directly as the attention map the Technical Agent consumes (Section 3.6), rather than re-deriving feature importance separately.

3.4.3 Contrastive Training

For each 60-candle window, a positive pair is two independently augmented views of the *same* window; all other windows in the mini-batch serve as negatives (standard in-batch-negatives InfoNCE). Four stochastic augmentations are applied independently: **amplitude scale** ($p = 0.5$, $\times$ Uniform$(0.85, 1.15)$), **jitter** ($p = 0.5$, i.i.d. Gaussian noise $\sigma = 0.03$), **channel mask** ($p = 0.5$, 2 random feature columns zeroed), and a **time warp** ($p = 0.3$, smooth sinusoidal resampling onto the original 60-step grid, amplitude up to 5 steps, analytically monotone so timestep ordering is never inverted). Figure 3.5 summarizes the full pretraining flow, from the two augmented views through the shared-weight encoder and projection head to the NT-Xent loss.

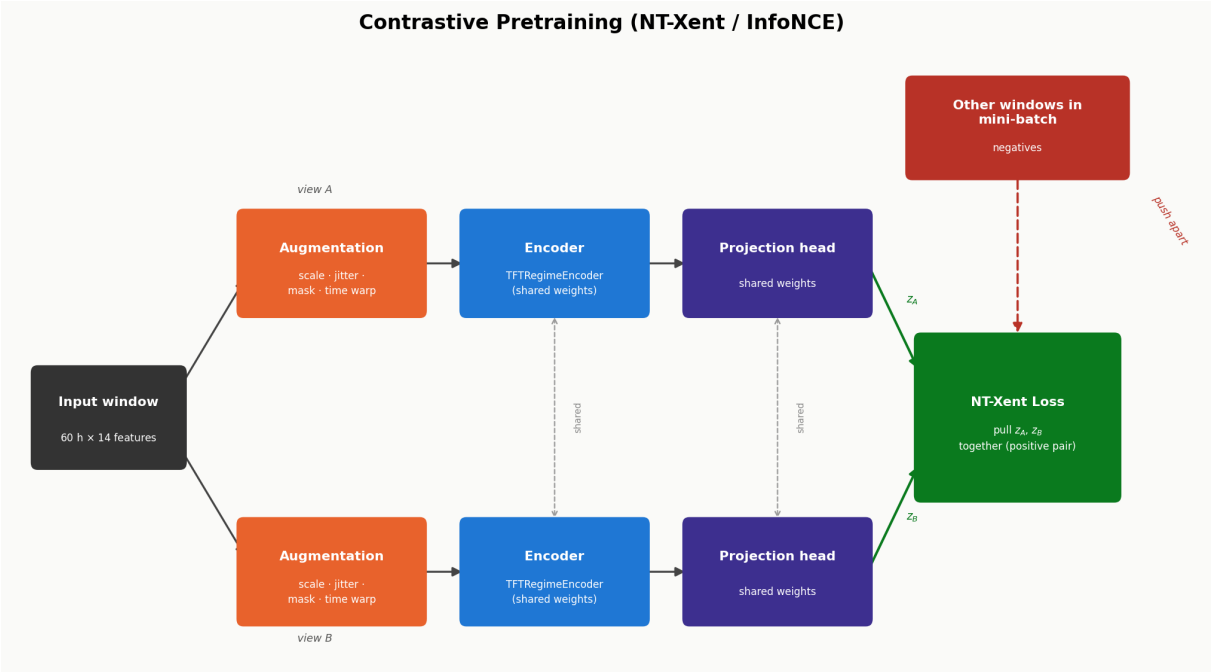


Figure 3.5: Contrastive pretraining of the regime encoder: two augmented views of the same window are encoded with shared weights and pulled together in embedding space, while other windows in the mini-batch act as negatives pushed apart by the NT-Xent loss.

The loss is NT-Xent: both views' L2-normalized embeddings, cosine similarity scaled by a temperature τ, cross-entropy against the identity matching. Temperature is learned rather than fixed, $\tau = \exp(-\ell)$ with $\tau_0 = 0.07$, so gradient descent can anneal similarity sharpness over training. An unconstrained ℓ was found to drift low enough to collapse the loss into a near-trivial objective during one run (Section 3.8); ℓ is therefore hard-clamped after every optimizer step so τ cannot leave $[\tau_{\min}, \tau_{\max}] = [0.03, 0.30]$.

Training runs only on the train split (Section 3.3.3), with 2024 validation loss driving early stopping. Three convergence gates are checked and saved inside the checkpoint itself, not only the external experiment tracker, so a remotely-trained checkpoint remains self-describing: validation loss < 4.0, top-1 retrieval accuracy > 0.20 (chance ≈ 0.002 at this batch size), and temperature inside $[0.03, 0.30]$.

3.4.4 Ongoing Refinement: Pooling and Pair Construction

A later diagnostic found the encoder's embeddings underperforming simple raw-feature baselines on short-horizon return predictability, pointing to two candidate causes rather than outright collapse: mean-pooling may discard recency information the LSTM's own hidden states already carry, and augmenting two views of the *same* window (rather than pairing it with a genuinely different, adjacent one) may teach an invariance unhelpful for near-term prediction. Two isolated experiments were run to test these independently: swapping mean-pooling for last-timestep pooling on the existing checkpoint (no retraining needed, both read the same LSTM hidden states), and a CPC-style [10] variant pairing each window with its immediately following, non-overlapping one instead of an augmented copy of itself. Neither had produced a conclusive result at the time of writing.

3.5 Regime Detection

3.5.1 Design Objective

Layer 2 discretizes market state into a small number of interpretable regimes with explicit transition dynamics, using a Gaussian Hidden Markov Model [2] rather than a non-sequential clustering method, specifically to obtain the persistence and switching structure that downstream position-sizing and agent-conditioning logic depend on. Sections 3.5.2–3.5.5 below describe the model as originally built, fitted over the continuous 128-dim embedding space produced by Layer 1's encoder (Section 3.4); Section 3.5.6 then describes the production revision, which fits the HMM directly on the 14 engineered features with no encoder or PCA stage.

3.5.2 Dimensionality Reduction

Before HMM fitting, the 128-dim embedding is reduced to 16 dimensions via PCA, fit exclusively on training-split embeddings to avoid any leakage of validation or test structure into the projection. The pipeline asserts that the retained components explain more than 75% of training-set variance before proceeding; the production fit retains approximately 85%.

3.5.3 Model Selection

A GaussianHMM is fit independently for each candidate state count $n \in \{2, \ldots, 8\}$ using diagonal covariance, 100 EM iterations, tolerance 10^{-3}, on a fixed 20,000-row subsample of the training embeddings (both fitting and scoring use the same subsample, so log-likelihoods are directly comparable across candidates). Each candidate is scored by BIC,

$$\mathrm{BIC}(n) = -2\,\mathrm{LL}(n) + \left(n^2 + 2n \cdot d\right) \ln(n_{\mathrm{train}}),$$

where $d = 16$ is the PCA dimensionality and the parameter count $n^2 + 2nd$ accounts for the transition matrix (n^2 entries) plus per-state diagonal mean and variance ($2d$ parameters per state). The candidate with the lowest BIC is not selected outright: a flat-curve guard instead selects the *smallest* n within 5% of the globally best BIC, on the reasoning that if several state counts fit the data almost equally well, the simplest one should be preferred. This guard was tightened during the project after an earlier version — which only compared the top two BIC candidates rather than every candidate against the global best — was found to produce a state count with duplicate, statistically indistinguishable states that never persisted more than a single hourly bar; an empirical audit combining KS/ANOVA distinctness tests with observed dwell times confirmed the corrected guard's selection was the genuine structural elbow in the BIC curve.

3.5.4 Final Fit and Regime Stream Generation

The selected state count is refit once more on the *full* training split (not the BIC-search subsample), with diagonal covariance, 200 EM iterations, tolerance 10^{-4}. Diagonal covariance is used for both the search and the final fit deliberately: an earlier version fit the final model with full covariance while the BIC parameter-count formula above still assumed diagonal covariance, a mismatch that let one state's covariance grow essentially unconstrained during EM, turning it into an "absorbing" catch-all state that any live, short-context HMM query collapsed into regardless of the actual input. Matching covariance structure between the BIC search and the deployed model closes that gap.

The fitted HMM and PCA transform are then applied to the *entire* dataset (train, validation, and test embeddings alike — inference through an already-fit model does not touch the sacred boundary the way fitting would) to produce a regime probability stream: for every asset and timestamp, a hard regime label (Viterbi-style MAP state) and a full probability vector over all n states, both stored for consumption by the agent layer and the RL environment. Figure 3.6 summarizes the production pipeline described in this and the preceding subsections — feature input through BIC-guided state-count selection, the best-of-5-seed final fit, and the resulting regime stream's two downstream consumers.

Not every regime label is populated for every asset: regime semantics are shaped jointly by all four assets' data during fitting, and some states (e.g. a stress/crisis state) are only ever visited by the forex/commodity series, which have a materially different volatility profile from crypto. Regime label semantics are also not guaranteed stable across refits of the HMM — the mapping from an integer label to an economic interpretation (e.g. "calm bull") is derived post hoc from each state's empirical statistics rather than fixed by construction, so any component that displays a human-readable regime description re-derives it from the current fit's own statistics rather than hardcoding a label-to-description table.

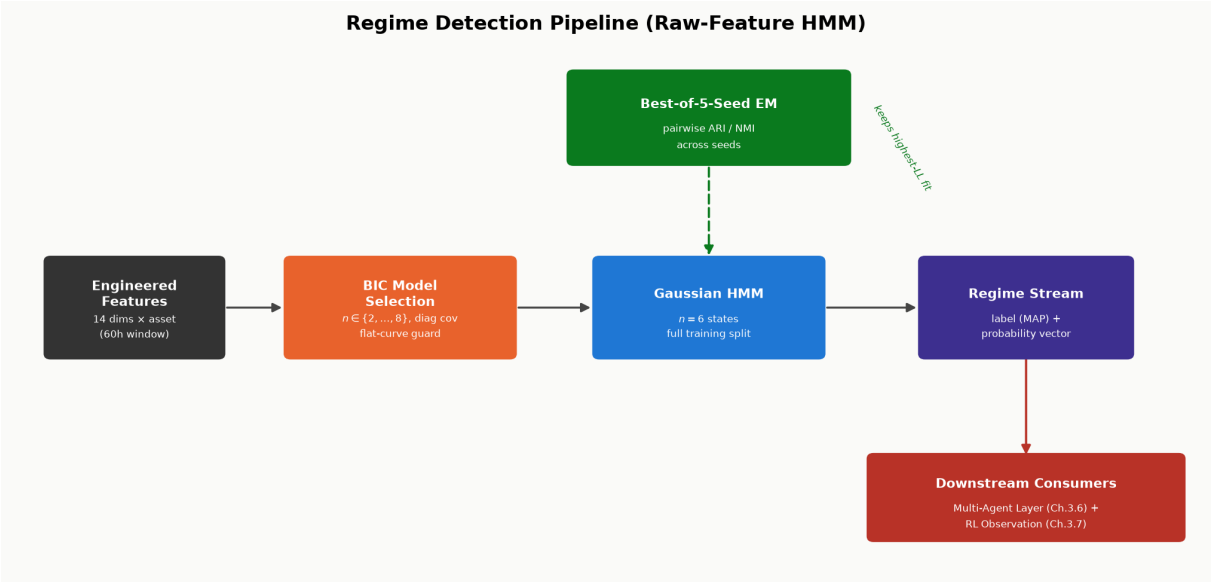


Figure 3.6: Regime detection pipeline: engineered features → BIC-guided state-count search → best-of-5-seed Gaussian HMM fit → regime probability stream, consumed by Layers 3 and 4.

3.5.5 Engine Correctness Audit

A dedicated audit of the regime-detection engine, conducted independently of the four-asset panel described above, found two correctness defects that predate this audit and had gone undetected because nothing in the pipeline checked for them. First, `fit_hmm_bic` subsampled 20,000 *random*, non-contiguous rows from the training embeddings before fitting each BIC candidate, sorted them back into chronological order, and fit the Baum–Welch algorithm on the result as though it were one continuous sequence. Two rows that are chronologically adjacent *in the subsample* can be hours or days apart in real time, which biases the estimated self-transition probability toward appearing to switch state far more often than the true chain does — exactly the failure signature (spurious short-dwell states) the flat-curve guard in Section 3.5.3 was built to catch, meaning the two defects compounded rather than being independent. Second, no call anywhere in the fitting or decoding path passed hmmlearn's `lengths` argument: every multi-asset embedding matrix is four assets' full histories concatenated back-to-back, so every fit and every Viterbi decode implicitly treated BTC's last training row as transitioning directly into ETH's first training row, and likewise at every other asset boundary.

Both defects were fixed at the source: the BIC search now fits every candidate on the full, correctly ordered training data (slower, not wrong), and a `lengths` array — computed once from the contiguous per-asset run structure that every embeddings file already has by construction — is threaded through every `.fit()`, `.score()`, and `.predict()` call in the module. In practice the boundary-crossing defect proved to have negligible numerical impact on the four-asset panel (three spurious cross-asset transitions out of well over 100,000 rows), but the subsampling defect was structural, not a rounding error, and is the more serious of the two.

Fixing this exposed a gap the original methodology had never checked: no seed-sensitivity analysis existed anywhere in the pipeline. The final refit was therefore changed to a best-of-five-seeds procedure (five independent EM runs from different initializations,

keeping the highest-log-likelihood model), with every fit now reporting pairwise Adjusted Rand Index (ARI) and Normalized Mutual Information across seeds' decoded labels. Applied to the production four-asset, four-state model, mean pairwise ARI across five seeds was only 0.193 (two seed-pairs statistically indistinguishable from chance) despite all five log-likelihoods sitting within 0.6% of one another — evidence that seed choice, not a well-identified clustering, had been determining the production labels (the deployed seed happened to be the best of the five, so this did not disqualify that specific checkpoint). A follow-up sweep found seed agreement improving monotonically as state count decreased (mean ARI 0.60 at $n = 2$, 0.42 at $n = 3$, 0.19 at $n = 4$), consistent with the four-state model over-asking the feature representation.

A separate implementation defect — a silent multiprocessing deadlock, found while parallelizing the five-seed refit across CPU cores — is recorded in Appendix A.2 rather than here, since it is a software-engineering diagnosis rather than a regime-detection methodology point. The fix was verified both directly (all five worker processes reaching the expected CPU utilization) and by a synthetic sanity check (Section 3.5.8) on pure random noise, which correctly returned near-chance seed agreement once the pool was actually computing.

3.5.6 Asset Panel Revision

Independently of the engine defects above, a raw-feature variant of the regime detector — fitting the Gaussian HMM directly on the 14 normalized features with no encoder and no PCA stage, motivated by the encoder's own inability to beat a raw-feature baseline on predictability (Section 3.4.4) — was audited against the four-asset panel of Section 3.3.1 and found to have collapsed onto asset identity rather than market regime. Cross-tabulating the fitted state labels against asset showed four of six states occurring almost exclusively for BTC/ETH and one state occurring in 96.5–98.5% of EUR/USD and XAU/USD's rows with a 0.998 self-transition probability (a 19.5-day mean dwell time, an order of magnitude longer than any other state); the maximum single-asset share of any state was 0.97–0.99. The mechanism is directly attributable to a property of the forex/commodity data documented in Section 3.3.1: because histdata.com carries no centralized volume figure, `obv_raw` and `vol_zscore` are literally constant, not merely low-variance, for EUR/USD and XAU/USD (confirmed independently when an unrelated stationarity check on the same two columns crashed outright with a "constant input" error). A diagonal-covariance Gaussian mixture fit on raw, unwhitened features is dominated by whichever input dimension separates the data most cleanly, and a column that is constant for exactly two of four assets and variable for the other two is close to a perfectly separating feature for asset identity — the encoder-and-PCA pipeline of Sections 3.4–3.5.2 does not exhibit this failure mode because PCA on a learned embedding does not carry the same raw per-feature scale asymmetry.

This finding motivated replacing the four-asset panel with a crypto-only, five-asset panel (BTC, ETH, BNB, LTC, XRP) for all raw-feature regime-detection work, removing the source of the degenerate feature entirely rather than attempting to repair it in place. Asset selection was constrained by data availability, verified directly against the exchange rather than assumed: BTC, ETH, BNB, and LTC all have genuine hourly data from 2017, while XRP's earliest available candle is 2018-05-04 (accepted, at the cost of the first four months of the panel's 2018 coverage) and SOL's is 2020-08-11 — a hard wall that would either truncate the entire panel's usable history to two and a half years or require a pipeline

redesign to support asymmetric per-asset start dates, and was excluded on that basis. Stablecoins were excluded from consideration on the same reasoning that ruled out the forex/commodity volume columns: a near-zero-variance price series would introduce the identical degenerate-feature risk from a different direction. Re-running the asset-collapse check on the resulting five-asset panel confirmed the fix: the maximum single-asset share of any fitted state fell to 0.234, close to the 0.20 a perfectly balanced five-asset panel would produce. The new panel was also run through the full pipeline validation described in Section 3.3.2, which required two corrections to the validation module itself — its asset-classification set had never been extended beyond the original two crypto symbols, and its report path was hardcoded in a way that would have overwritten the four-asset panel's own validation record — after which both the five-asset panel and a BTC-only single-asset panel (retained throughout as a comparison baseline, Section 4.1.1) passed with zero critical failures.

3.5.7 Empirical State-Count Validation at Scale

The five-asset and BTC-only panels inherited a state count of six from the raw-feature variant's own BIC search (Section 3.5.6), which is subject to the same subsampling defect described in Section 3.5.5 and had never been checked against either panel using the empirical method of Section 3.5.3. That method was re-run, generalized to accept an arbitrary feature source and state-count range, across $n \in \{3, \ldots, 8\}$ on both panels. Neither panel showed the duplicate-state or single-bar-dwell pathology the method was designed to catch, anywhere in the tested range: minimum inter-state mean distance fell smoothly from 2.80 at $n = 3$ to 1.60 at $n = 8$ on the five-asset panel (2.86 to 1.56 on BTC-only) with no discontinuity, and no state's mean dwell time dropped below two hours at any tested n. This is a materially different outcome from the four-asset, encoder-based model, where the same method had previously found a sharp elbow at $n = 7$; on the richer raw-feature representation, the sweep validates $n = 6$ as free of pathology without uniquely selecting it over any neighboring candidate. The six-state choice is retained on the strength of the downstream evidence gathered at that value (Section 4.1.1) rather than on the sweep alone, since the sweep by itself does not distinguish it from $n = 4$, $n = 5$, or $n = 7$.

3.5.8 Synthetic Ground-Truth Validation

Every validation described so far — distinctness testing, predictive-signal testing, seed-sensitivity reporting — is necessarily computed against real market data, for which the true underlying regime process, if one exists, is never observed. To obtain at least one setting in which recovery accuracy against a *known* ground truth could be measured directly, a synthetic six-state Markov-modulated regime-switching process was constructed and calibrated to the real, already-fitted five-asset model (matching its transition matrix and each state's empirical return/volatility directly), simulated with two independently implemented and cross-checked generators, and run through the unmodified production feature-engineering and HMM-fitting pipeline to score recovered labels against the known truth. The generator's construction, its two discretization schemes, and three independent numerical cross-checks of its calibration are given in full in Appendix A.1.

Five candidate explanations for the resulting recovery level were tested directly rather than assumed, each isolating one variable: more calendar time per path, the real best-of-

five-seed fitting procedure in place of a single seed, pooling multiple independent synthetic assets, pooling assets with a verified shared regime correlation, and matching production's real training-set scale exactly. None moved recovery accuracy outside a narrow Adjusted Rand Index (ARI) band of 0.15–0.20, ruling out data quantity, cross-asset correlation, and seed count as explanations.

The eventual explanation was obtained directly: a Bayes-optimal classifier built from the *true* generating parameters — the best possible single-observation classification given perfect knowledge of the answer — achieves an ARI of only 0.047, below every empirically achieved value, because the six states' hourly log-return means are all near zero and of each other, and their volatility differences overlap substantially at single-observation resolution. Distinguishing six heavily overlapping, near-zero-mean Gaussians from single noisy draws is close to an irreducible classification problem regardless of estimation quality. This is consistent with, not contradicted by, the seed-sensitivity finding of Section 3.5.5: seed agreement measures whether independent fits converge to the same *parameters* (which more data improves), while this recovery test measures per-timestep classification accuracy, which is bounded by class overlap and unaffected by data quantity. A model can be perfectly reproducible across refits while remaining irreducibly noisy at the per-timestep level.

3.6 Multi-Agent Deliberation System

3.6.1 Design Objective

Layer 3 conditions a structured, multi-perspective reasoning process on the regime signal from Layer 2, addressing the interpretability gap identified in Section 2.4.3: rather than a single LLM call producing an opaque recommendation, five specialized agents each analyze the current market state from a distinct angle, structured as an explicit two-round debate synthesized by a final decision-making agent — the multi-agent debate pattern motivated in Section 2.4.2.

3.6.2 Orchestration and LLM Backend

The debate is orchestrated as a LangGraph state machine. Every agent call goes through a shared base class that wraps a single local LLM, Mistral 7B Instruct served via Ollama with GPU acceleration, called with temperature 0.4, a 600-token generation budget, and the endpoint's native JSON-constrained output mode, under a 300-second timeout with one retry on failure. Because a locally-hosted 7B model does not reliably emit clean JSON even in constrained-output mode, every LLM response passes through a five-stage recovery procedure before being treated as unparseable: (1) direct Pydantic validation of the raw response, (2) extraction from a markdown code fence, (3) regex extraction of the first JSON object anywhere in the text, (4) a greedy first-brace-to-last-brace extraction with trailing-comma repair, and (5) the same greedy extraction with embedded literal newlines collapsed before repair. Only if all five stages fail is a safe, neutral default substituted — and that fallback is always logged, so a systematic parsing failure is visible rather than silently corrupting a downstream decision.

3.6.3 Regime Context

Every agent call is grounded in a shared `RegimeContext` object built once per debate: the regime label and probability vector for the queried asset and timestamp (from Section 3.5), the corresponding market feature window, and any upcoming high-impact macroeconomic events. This ensures all five agents reason about the identical regime signal rather than each re-deriving their own, inconsistent notion of "current conditions."

3.6.4 Agent Roster

3.6.5 Debate Flow

The debate proceeds in three stages. **Round 1**: Macro, Technical, and Sentiment agents each independently produce a thesis (directional bias, confidence, up to four supporting factors, and an assessment of whether their view is aligned, contrary, neutral, or mixed with respect to the current regime); RiskAgent computes its risk limits in parallel, deterministically, with no LLM call. **Contrarian check**: the Contrarian agent evaluates the Round-1 average confidence and, if the 0.60 threshold is met, produces a forced-opposite thesis with historical grounding. **Round 2**: rebuttals are triggered under any of three conditions — the Contrarian is active (which guarantees at least one point of opposition), two Round-1 agents already hold directly opposing biases, or Round-1 shows high average confidence (> 0.75) with no internal disagreement at all (a groupthink check). When none of these hold, Round 2 is skipped entirely and no rebuttals are produced. When active, each participating agent may either maintain or revise its bias and confidence in light of the others' arguments, and the Contrarian itself posts a rebuttal targeting whichever opposing agent holds the highest confidence. Figure 3.7 summarizes this flow end to end, distinguishing the three LLM-based directional agents (blue) from the deterministic RiskAgent (orange, no LLM call) and the two conditional stages (Contrarian, Round 2) from the always-executed Round 1 and Fund Manager synthesis stages.

3.6.6 Risk Limit Computation

RiskAgent fits a GARCH(1,1) model on the trailing 90 days of hourly returns for the queried asset (falling back to the rolling sample standard deviation if the GARCH fit fails to converge), producing a one-step-ahead volatility forecast $\hat{\sigma}$. Value-at-risk and Conditional Value-at-Risk at the 95% level are then computed under a normal-tail approximation,

$$\mathrm{VaR}_{95} = \Phi^{-1}(0.05)\,\hat{\sigma}, \qquad \mathrm{CVaR}_{95} = -\frac{\phi(\Phi^{-1}(0.05))}{0.05}\,\hat{\sigma},$$

with Φ the standard normal CDF and ϕ its density. The maximum position size is capped so that a 2%-of-capital daily CVaR budget is never exceeded, $\text{max_position} = \min(1, 0.02/|\mathrm{CVaR}_{95}|)$, and a coarse risk-regime label (low/medium/high/extreme) is assigned from $\hat{\sigma}$ thresholds. This position cap is throttled further, independently of the LLM agents, ahead of scheduled high-impact macroeconomic releases (e.g. FOMC, CPI, NFP): to 50% of its computed value inside a 6-hour window before the event, or 75% inside 24 hours, so that event risk is capped by a hard, auditable rule rather than left entirely to whatever the agents happen to notice in their prompt context.

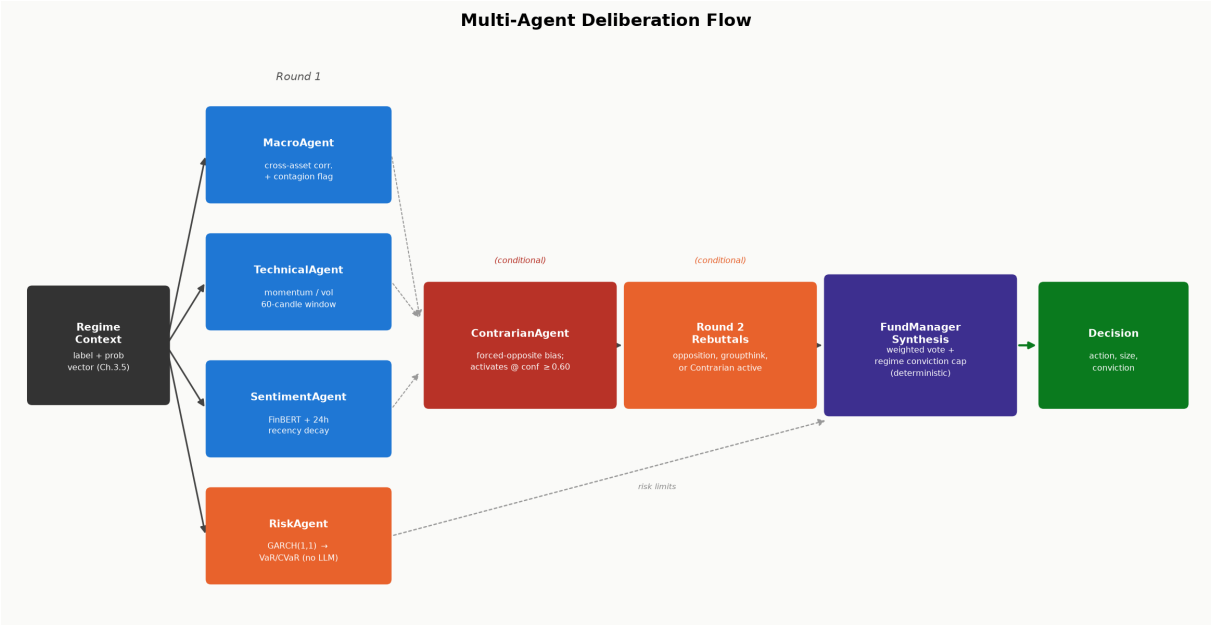


Figure 3.7: Multi-agent deliberation flow: Round 1 (three LLM directional agents plus the deterministic Risk agent), the conditional Contrarian check and Round 2 rebuttals, and the deterministic Fund Manager synthesis producing the final decision.

3.6.7 Fund Manager Synthesis

The FundManager computes a weighted consensus from all Round-1/Round-2 outputs: each agent's final (post-rebuttal, if any) confidence is signed by its directional bias and summed into a bullish score and a bearish score, with the Contrarian's contribution counted at half weight throughout (reflecting its role as a deliberate outlier check rather than an equal vote). Consensus is declared bullish or bearish only if one side exceeds the other by a margin of 0.15; otherwise the debate is recorded as split. A regime-dependent conviction multiplier is then applied — for instance, a bullish consensus is scaled to 40% conviction in a stress/crisis regime and 50% in a bear-trend regime, while a bearish consensus in a calm-bull regime is scaled to 70% — and if the regime unambiguously contradicts the voted direction (bullish consensus during stress/crisis or a bear trend), the decision is overridden to `hold` entirely rather than merely down-weighted. A second, independent override condition fires whenever the Contrarian was active *and* at least one other agent changed its bias after the Round-2 rebuttal — treating a debate that visibly moved an agent's position as evidence the regime override should be flagged even without a hard regime contradiction. The final position size is $\min(0.8 \times \text{conviction},\ \text{max_position_size})$ from RiskAgent, and the FundManager's own LLM call is used only to produce a one-sentence natural-language rationale for the already fully-determined numerical decision — the action, size, and override flags are all computed deterministically before the LLM is consulted, so a parsing failure on the rationale text cannot corrupt the decision itself.

3.7 RL Execution Agent

3.7.1 Design Objective

Layer 4 translates the Layer 3 debate output, together with the regime embedding and current portfolio state, into a concrete position-sizing decision, learned end-to-end with reinforcement learning rather than a fixed sizing rule, so that risk exposure can be modulated by both regime and the debate's own confidence.

3.7.2 MDP Formulation

State: the market-state block was originally the 128-dim embedding from the frozen Layer-1 encoder (no gradient flows back into it during RL training), giving a 156-dim observation overall. After the encoder was found not to outperform engineered features on predictability (Section 3.4.4) and superseded (Section 3.5.6), production observations instead flatten the raw 60-candle engineered-feature window directly — $60 \times 14 = 840$ dimensions on the original four-asset panel, or $60 \times 22 = 1320$ on the crypto5 panel, which carries 8 additional multi-horizon-normalization columns — optionally concatenated with the corresponding HMM regime-probability vector when regime-conditioning is being tested explicitly (`obs_mode="raw_regime"`; 6 additional dimensions on the crypto5 panel's six-state model). In every variant this market-state block is followed by: a 16-dim agent-consensus vector (per-agent signed confidence, the FundManager's signed conviction, position size, regime-override flag, and dissent count — zeroed when no cached debate exists for the current timestamp, the common case during training, since the debate system is never invoked live during RL training or rollout); a portfolio-state block of $N{+}4$ dimensions, one canonical signed position slot per active asset ($N = 4$ on the original panel, $N = 5$ on crypto5) plus normalized running PnL, drawdown, steps-since-last-trade, and the current GARCH volatility forecast; and an N-dim one-hot asset identifier. The one-hot block was added after a multi-seed audit found that, without it, the shared policy's per-asset behavior was unstable immediately after a curriculum phase introduced a second asset: the only asset-identifying signal otherwise available to the network was the portfolio position slot, which reads as zero at the start of every episode and gives the network no reliable way to know which asset it is trading until it has already taken a position. The full observation vector is clipped to $[-10, 10]$ in every mode (e.g. 868-dim total for the original four-asset panel's raw-feature checkpoints, 1350-dim for crypto5); the encoder-based, 156-dim variant remains implemented and selectable (`obs_mode="encoder"`) but is not the one used to produce the checkpoints evaluated in Chapter 4.

Action (hybrid): a discrete direction $\in \{\text{long}, \text{short}, \text{hold}\}$ sampled from a Categorical head, and a continuous position fraction $\in (0, 1)$ sampled from a Beta distribution whose shape parameters are constrained to $\alpha, \beta > 1$ via a softplus transform, keeping the distribution unimodal so it cannot collapse mass onto the 0/1 boundary the way an unconstrained Beta could. The Beta distribution was chosen specifically because it is natively bounded on $(0, 1)$, so the position fraction requires no post-hoc clipping and carries no boundary-gradient bias the way a clipped Gaussian head would.

Reward:

$$r_t = \frac{\Delta \text{PnL}_t}{\text{CVaR}_t} - \lambda \cdot \text{drawdown}_t - \text{idle_penalty}_t,$$

where CVaR_t uses the identical Cornish–Fisher-style normal-CVaR formula as RiskAgent (Section 3.6.6), computed from a GARCH(1,1) forecast that is refit on a weekly stride and

forward-filled between refits (never refit per environment step, which would make training intractable), and reward is clipped to $[-10, 10]$ to keep gradients well-scaled for the value network. The drawdown penalty is continuous from the first bar of drawdown rather than gated behind a threshold, since a threshold gate leaves small, common drawdowns with zero training signal. The idle penalty is a small constant deduction applied whenever the agent holds a near-zero position, added specifically to break an "always-flat is the safest optimum" local minimum the policy otherwise converges to (Section 3.8); it is a training-time-only signal and is always set to zero during validation and test evaluation, so that abstention is never artificially discouraged when measuring genuine trading performance.

3.7.3 Policy Network

The actor-critic policy shares one MLP trunk ($156 \to 256 \to 128 \to 64$, ReLU activations) feeding three independent linear heads: a 3-way Categorical logits head for direction, a 2-way head producing the Beta distribution's pre-softplus (α, β) parameters, and a scalar value head. This required overriding Stable-Baselines3's default policy-construction and action-distribution machinery directly, since SB3's built-in PPO implementation supports only a single, homogeneous action distribution and has no native support for a hybrid discrete/continuous action space. Figure 3.8 summarizes the full observation-to-reward loop described in this and the preceding subsection: the shared trunk, the two action heads, the resulting hybrid action, the environment step, and the reward signal that produces the next observation (the value head is omitted from the figure for clarity, since it has no role in the action-selection path shown).

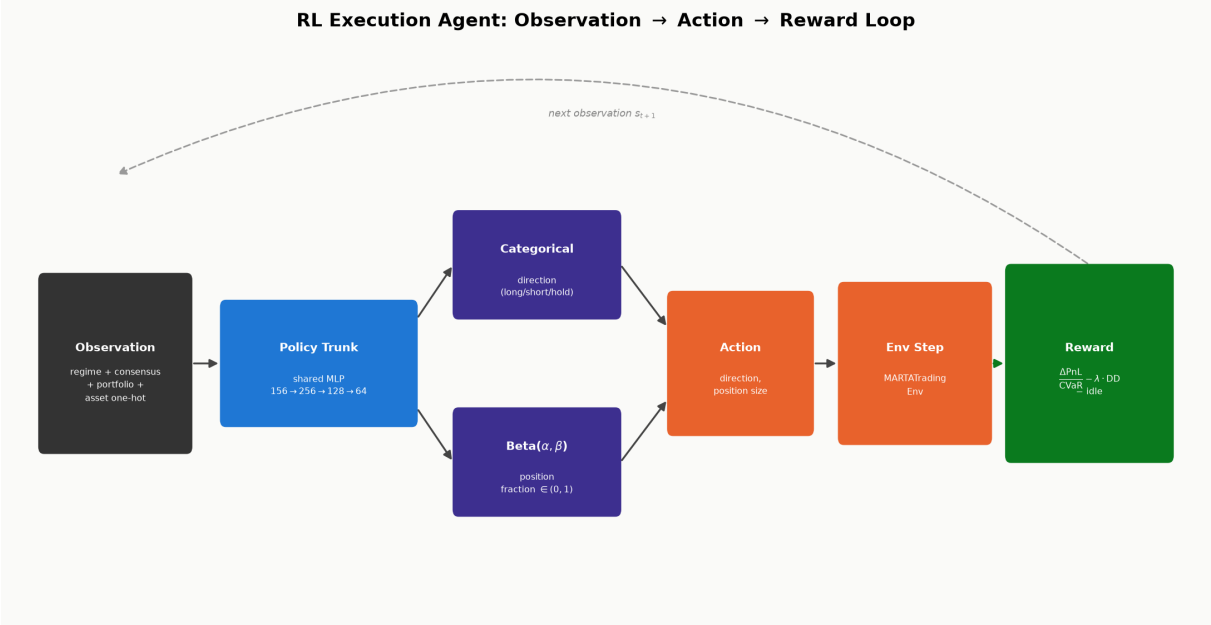


Figure 3.8: RL execution agent: the shared policy trunk feeds a Categorical direction head and a Beta position-fraction head, whose combined output is executed by the environment and scored by the CVaR-normalized reward, producing the next observation.

3.7.4 Curriculum Training

This subsection describes the original curriculum design, trained on the initial four-asset panel of Section 3.3.1. It predates, and is a different design from, the native weekly-decision-cadence PPO training later run on the production five-asset crypto-only panel of Section 3.5.6 (Chapter 4, Section 4.3.4), which does not use a phased curriculum at all.

Training follows a curriculum, motivated by the general practice (Section 2.5.3) of mastering a single, liquid asset before extending to less liquid or structurally different ones. Four phases are defined — BTC only; BTC+ETH; +EUR/USD; +XAU/USD — each with its own validation-Sharpe gate that must be met before the shared policy is allowed to advance, and the environment's active asset list is mutated in place between phases rather than restarting training, so that policy weights (and therefore any cross-asset transfer) persist across the whole curriculum. In practice, the two forex/commodity phases were found to actively erode the BTC/ETH policy learned in the first two phases: because histdata.com's underlying intraday bars carry comparatively little genuine hourly signal relative to the crypto pair, "hold" is close to the reward-minimizing-variance action for those assets, and continuing training into Phases 3–4 pulls the shared policy toward an always-flat behavior that also degrades its BTC/ETH performance. Production training therefore caps the curriculum at Phase 2 (BTC+ETH) by default; Phases 3–4 remain fully implemented and are used only for the explicit ablation and diagnostic runs described in Section 3.8, not for the headline checkpoint.

Eight parallel environments, each starting from an independently randomized start point within the active split, are used specifically to decorrelate the rollout buffer: a single environment produces one long autocorrelated trajectory per buffer, which was found to destabilize training, whereas 8 independent trajectories give PPO a buffer closer to the i.i.d. assumption its advantage estimates rely on. The discount factor was lowered from an initial 0.99 (an effective $\sim$100-step horizon) to 0.95, and the entropy coefficient is annealed from a comparatively high 0.05 down to 0.005, both changes made after diagnosing that a lower-entropy, longer-horizon initial configuration collapsed too quickly to an always-hold policy before it could discover profitable trading behavior.

3.7.5 Evaluation Determinism

Because the validation window contains relatively few meaningful trades, small sources of non-determinism were found to be sufficient to flip the sign of the measured Sharpe ratio between otherwise-identical evaluation runs. Two concrete sources were identified and fixed: the frozen encoder's forward pass ran on GPU (unseeded cuDNN kernels) once per environment step during evaluation, and neither PyTorch nor NumPy's global RNG was seeded before an evaluation pass. All evaluation-only environments and model loads are now forced onto CPU (with the shared encoder module deep-copied before being moved, since moving it in place would otherwise silently drag a live training environment's encoder onto CPU too), and both RNGs are seeded immediately before every evaluation pass. Repeated evaluation runs on an identical checkpoint now return bit-identical metrics.

3.8 Evaluation Methodology

3.8.1 Metrics

Trading performance is summarized by the annualized Sharpe ratio (hourly returns annualized by $\sqrt{24 \times 365}$), maximum drawdown, and Calmar ratio, alongside execution-level diagnostics: because the continuous Beta position head produces a nonzero-sized change on almost every step, a raw trade count is not a meaningful measure of trading frequency, so a trade is only counted as *meaningful* once the position change exceeds 0.5% of capital, and mean turnover is reported separately as the actual measure of execution activity.

3.8.2 Ablation Design

The central empirical question of this work — whether regime-awareness measurably improves risk-adjusted trading performance — is answered through a seven-variant ablation isolating the contribution of the regime embedding, the multi-agent consensus, and the PPO execution layer independently:

Each variant's result records which of these methods actually produced it, so that, for instance, a reader comparing `ppo_no_regime` against `marta_full` is comparing "momentum execution without regime context" against "the trained policy with regime context," not silently comparing against a degenerate always-flat run mislabeled as a fair baseline.

3.8.3 Statistical Testing

Statistical significance of the Sharpe-ratio delta between `marta_full` and each baseline is assessed with a block bootstrap (1000 resamples, fixed seed) rather than an i.i.d. bootstrap over individual hourly returns: returns are resampled in non-overlapping 168-hour (one-week) blocks, preserving the autocorrelation structure of the underlying return series, which an i.i.d. resampling scheme would otherwise understate. The reported delta is the bootstrap mean of the per-resample Sharpe difference, with the 2.5th/97.5th percentiles reported as a 95% confidence interval; returns are trimmed to whole blocks before resampling.

3.8.4 In-Sample Regime Behavior Diagnostic

A separate, explicitly diagnostic-only probe examines how the trained agent sizes positions across known crisis and bear episodes that fall inside the *training* split — the March 2020 crash and the 2018 crypto bear market — reporting average position size per decision-time regime and per-regime-transition position changes. Because both windows are in-sample, this analysis is never combined with, or presented alongside, the out-of-sample validation/test metrics above; it answers a qualitative question (does the policy's behavior differ meaningfully across regimes it was trained on) rather than a performance claim.

3.9 Dashboard and Live System

3.9.1 Design Objective

Layer 5 exposes regime detection, agent reasoning, and execution decisions from Layers 1–4 in a live, auditable interface, directly supporting the interpretability property identified

as largely absent from prior work in Section 1.3.1.

3.9.2 Architecture

The backend is a FastAPI application; the frontend is a Next.js single-page application. Scheduling is handled by APScheduler, running an hourly cron-triggered signal tick rather than a message-queue-based task system, since the system's decision cadence (one debate and one execution decision per asset per hour, matching the pipeline's native hourly resolution) does not require queue-based task distribution. Real-time price, regime, and debate state are pushed to the frontend over WebSocket connections; trade execution runs against Alpaca's paper-trading API, with Telegram used for out-of-band alerting and a chat interface for ad hoc natural-language queries against the live system state.

The dashboard serves both the crypto5 production panel (Section 3.5.6) and the original four-asset panel side by side, the latter retained for its longer live-trading track record rather than removed once the panel migration was adopted for new development. Per the RL ablation's outcome (Section 3.8.2; Chapter 4), the hourly signal tick's default execution path applies the multi-agent debate's own risk-capped position size directly rather than routing it through the trained PPO policy; PPO execution remains available only as an explicit, opt-in toggle in settings, surfaced with a visible warning that references the ablation finding it did not overcome.

3.9.3 Live Regime Lookup

Regime lookup exists in two distinct forms that are deliberately not interchangeable. A static lookup resolves the nearest row in the pre-computed regime probability stream (Section 3.5.4) for a given historical timestamp, which is correct and cheap for genuinely past bars but is a frozen snapshot that stops advancing the moment the offline regime pipeline last ran. A live lookup instead recomputes PCA and HMM inference fresh from a rolling 168-hour embedding sequence ending at the query time; a single-frame HMM query was tried first and found to be dominated by the model's stationary prior rather than the actual current observation, which is why a full sequence, not a single point, is required for a live query to actually move. Every component that reasons about "now" (the live price feed's regime tag, the chat assistant, the hourly signal tick) uses the live lookup; only genuinely historical queries use the static one.

Both panels' regime services expose this same static/live distinction independently — the crypto5 panel's six-state model and the legacy panel's four-state model are two separately fitted HMMs, each with its own static stream and its own live-recomputation path, rather than one shared implementation silently assuming a fixed state count. A concrete failure mode of the static lookup surfaced after the crypto5 migration: nearest-neighbor lookup has no upper bound on how far a query timestamp may sit past the stream's last row, so every query issued after that row resolves to it indefinitely, and once the stream's last recorded row was months in the past this manifested in production as a live price chart showing one repeated regime label across its entire visible window rather than a single stale bar. The fix — a live-aware series lookup that prefers the nearest hit in the incrementally-growing live regime log (populated continuously by the live path above once the system is running) over the frozen static stream, falling back to a fresh live computation only when no logged entry exists within a bounded recency window — had to be applied at every call site independently rather than once centrally: an initial fix

to the historical-data endpoint left the real-time chart's own history-backfill path on the old unbounded lookup, and the same symptom recurred there until it too was migrated.

A related but distinct defect in the same live-facing text was that regime descriptions used directional language ("bull"/"bear"/"crisis") derived from each state's historical Sharpe ratio; Section 4.1.5 shows this language does not reliably track actual price direction and replaces it with a direction-agnostic volatility/trend-strength description, which every live-facing regime label — chart legend, agent prompt context, and dashboard description endpoint alike — now uses instead.

3.9.4 Live Feature Bridge

This bridge is specific to the legacy four-asset panel's two traditional-finance instruments; the crypto5 production panel's five assets are all fetched continuously via the same CCXT source used at training time and therefore need no equivalent bridge. Because EUR/USD and XAU/USD's training-time data is capped at 2025-12-31 (Section 3.3.1), a separate live feature bridge pulls real hourly OHLC bars from a market-data API and reproduces the exact same feature-engineering and normalization functions used at training time (Section 3.3.2), rather than a parallel reimplementation, appending them onto an append-only raw-bar accumulator that is itself never passed back through feature computation. This last detail matters operationally: an earlier version fed its own already-warmup-trimmed output back in as the next cycle's context, losing tens of rows to indicator warm-up against a single new bar per hourly cycle, which erodes the usable history to nothing within days; keeping a separate, untouched accumulator avoids that failure mode. BTC and ETH's own training-time data reaches materially further forward already and do not yet have an equivalent bridge, since the resulting staleness for those two assets is measured in weeks rather than months.

3.9.5 Multi-Asset Debate Coverage

The multi-agent debate cache backing the dashboard's cached-consensus signal (Section 3.6.2) initially had dense, real (non-mocked) coverage only for BTC, inherited from the original four-asset panel's development history; the other four crypto5 assets had at most a handful of debates each. Once the crypto5 panel became the production target, a batch generation pass produced several hundred real daily-stride debates per asset spanning the full 2024 validation window for every one of the five assets, so that the dashboard's live cached-consensus signal — and the `agents_no_regime` ablation variant of Section 3.8.2 — draw on comparable, asset-independent coverage rather than a BTC-only cache silently determining most of what any other asset's live signal shows.

3.9.6 User-Configurable Strategy Backtesting

Beyond the fixed ablation study of Section 3.8.2, the dashboard exposes a Strategy Tester panel letting a user choose an asset, a strategy, and an arbitrary date range and receive a real backtest — equity curve against a frictionless buy-and-hold reference, Sharpe/drawdown/win-rate summary statistics, and a trade log — computed on demand rather than read from a precomputed results file. Every indicator the panel's rule-based strategies need (moving averages, RSI, MACD, Bollinger width, and similar) is recomputed directly from price at request time rather than read from the pipeline's

own precomputed indicator columns, since those columns are only populated up to each asset's static-file boundary (Section 3.3.1) and are undefined on the live-extended rows a user-chosen range reaching the present would otherwise need. Alongside a set of purely rule-based technical strategies, the panel also exposes the native weekly-cadence PPO checkpoint of Section 4.3.4, in both its regime-conditioned and regime-free forms, letting a user reproduce that section's backtest interactively over a self-chosen window rather than only reading its reported figures.

3.9.7 Auditability

Every quantity displayed alongside a trading decision — win rate, best/worst trade, position history — is sourced from the same evaluation run that produced the headline Sharpe ratio shown next to it, rather than a separately generated snapshot that could silently drift out of sync with the number it is displayed beside; this was a deliberate design correction after an earlier version combined numbers from two different data sources on the same dashboard card.

The same on-demand backtesting capability introduced a distinct auditability risk once it covered a trained model rather than only rule-based strategies: nothing prevented a user from choosing a date range for the weekly-cadence PPO strategies that reached back into the checkpoint's own training window, and doing so produces a backtest that looks like a genuine result while actually re-evaluating the policy partly on data it was fit to. The panel now attaches a visible warning to any such request whose start date precedes the test-split boundary (Section 3.3.3), so a large but spurious in-sample result cannot be mistaken for the walk-forward evaluation reported in Chapter 4 without the tool itself flagging the discrepancy; rule-based strategies, which are never fit to any window, carry no such warning.

Table 3.2: Regime encoder architecture (`TFTRegimeEncoder`)

Stage	**Shape**	**Description**
Input	$(B, 60, 14)$	60-hour window × 14 normalized features
VSN gate	$(B, 60, 14)$	Linear(14→14) + softmax over the feature axis at every timestep, elementwise-multiplied with the input — a learned, per-timestep feature-importance weighting.
LSTM encoder	$(B, 60, 128)$	2-layer `nn.LSTM`, hidden size 128, inter-layer dropout 0.1
Mean pool	$(B, 128)$	Unweighted mean over the 60 timesteps — chosen for robustness to regime length over, e.g., last-timestep pooling.
Gated residual	$(B, 128)$	$h \odot \sigma(\text{Linear}(h))$ — a sigmoid gate applied to the pooled representation, mirroring TFT's gated residual connections. In this original design, this 128-dim vector was the embedding consumed downstream (Sections 3.5–3.7); those sections now describe both this original path and the production revision that bypasses it.
Projection head	$(B, 64)$	Linear(128→128) + ReLU + Linear(128→64) — used only by the contrastive loss during pretraining and discarded at inference time, following standard contrastive-learning practice (the projection head is known to distort the representation in ways that help the contrastive objective but do not help downstream tasks, so only the pre-projection embedding is kept).

Table 3.3: Encoder training hyperparameters

Parameter	**Value**	**Parameter**	**Value**
Window size	60 h	Batch size	512
Learning rate	1×10^{-3}	Weight decay	1×10^{-4}
Max epochs	200	Early-stopping patience	20 epochs
Warm-up	10 epochs	Gradient clip (max norm)	1.0

Table 3.4: The six agents of Layer 3 (five LLM-based deliberation agents plus one deterministic risk-limit agent)

Agent	**Role**
MacroAgent	Cross-asset correlation matrix and contagion flag computed over a 168-candle (7-day) lookback window; LLM interprets the resulting statistics into a directional thesis.
TechnicalAgent	Momentum and volatility indicators over a 60-candle window. An earlier version additionally consumed the regime encoder's VSN attention weights (Section 3.4.2) as a supplementary feature-importance signal; this was removed once the encoder itself was superseded (Section 3.5.6), since the checkpoint that produced those weights predated the crypto-only panel this agent now reasons over and was never trained on it.
SentimentAgent	FinBERT sentiment scores over news headlines within a 24-hour lookback window, aggregated with exponential recency decay (half-life 6 h); a 6-hour-vs-24-hour momentum comparison is also computed. FinBERT produces the quantitative scores; the LLM call interprets them into a narrative thesis, it does not re-score sentiment itself.
RiskAgent	The only non-LLM agent: fits a GARCH(1,1) model on the lookback window's hourly returns, converts the resulting one-step-ahead volatility forecast into 95% VaR/CVaR under a Gaussian-tail assumption, and derives a hard maximum position size and leverage cap (Section 3.6.6).
ContrarianAgent	Devil's-advocate agent; activates only when the Round-1 average confidence across the other agents reaches 0.60. When active, its `directional_bias` is *forced* to the opposite of the emerging consensus regardless of what the LLM itself argues, and it retrieves historical analogues — past occurrences of the same regime label with probability > 0.80 — reporting the empirical loss rate and average 24-hour forward return across up to 100 such cases, to ground its counter-argument in data rather than rhetoric alone.
FundManagerAgent	Synthesizes the full debate (Round-1 theses, Round-2 rebuttals, risk limits) into a single final decision (Section 3.6.7); the only agent whose output is actually executable.

Table 3.5: PPO training configuration

Parameter	**Value**	**Parameter**	**Value**
Learning rate	3×10^{-4}	Parallel environments	8
Rollout length (per env)	512	Rollout buffer size	4096
Minibatch size	64	Epochs per update	10
Discount γ	0.95	GAE λ	0.95
PPO clip range	0.2	Value-loss coefficient	0.5
Entropy coefficient	$0.05 \rightarrow 0.005$ (decayed over 100k steps)	Max gradient norm	0.5
Drawdown penalty λ	0.01	Max steps/episode	2000

Table 3.6: Ablation variants

Variant	**Regime / Agents / PPO**	**Method**
`buy_hold`	— / — / —	Static long-only baseline.
`rsi_strategy`	— / — / —	Rule-based RSI(14) mean-reversion (overbought 70 / oversold 30).
`ma_crossover`	— / — / —	Rule-based moving-average(12,26) crossover.
`ppo_no_regime`	— / — / PPO	Zeroing the 128-dim regime slot pushes the trained policy out-of-distribution and it collapses to constant-hold, which would trivially read as "identical to hold" rather than isolating regime dependence; this variant instead runs a 3-bar unanimous-momentum rule at fixed 0.3 position size, representing "RL-like execution without regime awareness."
`agents_no_regime`	— / Agents / —	Rule-based on the cached multi-agent debate consensus, using nearest-debate interpolation (snapping each step to the closest cached debate within 7 days) since the debate cache is sparse relative to the evaluation window; FundManager position size is capped at 5% to keep this variant's sizing comparable in scale to PPO's own risk-capped output.
`ppo_agents_no_regime`	— / Agents / PPO	As `ppo_no_regime`, with position size additionally scaled by the nearest cached debate's confidence.
`marta_full`	Regime / Agents / PPO	The complete system.

Chapter 4

Results

This chapter reports the outcomes of the evaluation protocol described in Section 3.8, following the same order as the methodology chapter. Section 4.1 reports the regime-detection layer's validation results in full, since that layer's methodology (Sections 3.5.5–3.5.8) was substantially revised over the course of this project and its results directly informed how the layers built on top of it are interpreted. Section 4.2 reports a dedicated audit of the multi-agent deliberation layer (Section 3.6), tracing every agent from prompt to final action rather than assuming the architecture description implies the behavior. Section 4.3 reports the RL execution layer's results (Section 3.7): a controlled ablation on the crypto-only panel that produced this project's final decision on that layer, and a subsequent follow-up experiment that reopened, without resolving, part of that question.

4.1 Regime Detection Results

Six regime-detection configurations were carried through the full validation battery: the original four-asset, encoder-based production model; the five-asset crypto-only panel introduced in Section 3.5.6, in both its 14-column production-feature form and a 22-column form extended with 90-day and 180-day rolling-normalization horizons alongside the production 720-hour window; and the same two feature variants restricted to BTC alone, retained throughout as a single-asset comparison baseline.

4.1.1 Statistical Validity Across Candidate Configurations

Two findings hold across all six configurations without a single exception. Volatility distinctness (`realized_vol_raw`, ANOVA across states, tested per asset independently) is significant at $p \approx 0$ in every configuration and every asset — the discovered states are genuine, statistically real volatility clusters regardless of panel or feature-horizon choice. Next-hour directional predictive signal from the regime label alone is chance-level (AUC 0.49–0.52) in every configuration, and adding the regime label on top of the raw features never measurably improves on the raw features alone.

A trade-off not visible from either extreme alone emerged once every feature-horizon variant had been tested against the same battery: the 22-column feature set gives near-perfect seed stability (0.989–1.000) at the cost of weaker return distinctness (2/5 assets significant versus 4/5 for the 14-column set on the five-asset panel), and neither variant's return-distinctness advantage translates into predictive AUC. The 22-column set is

Table 4.1: Seed stability and distinctness across all six configurations tested. Seed ARI is the mean pairwise Adjusted Rand Index across five independent EM restarts (Section 3.5.5). `close_return` / `realized_vol_raw` distinctness report the outcome of a per-asset ANOVA across states on that quantity.

Configuration	**States**	**Seed ARI**	`close_return` **distinctness**	`realized_vol_raw`
Production (4-asset)	4	0.193	weak, 1–3/6 pairs indistinguishable per asset	$p \approx 0$, all
5-asset, 14-col	6	0.585	4/5 assets significant, 0 indistinguishable pairs	$p \approx 0$, all
5-asset, 22-col	6	**1.000**	2/5 assets significant, 0 indistinguishable pairs	$p \approx 0$, all
BTC-only, 14-col	6	0.698	not significant, 0 indistinguishable pairs	$p \approx 0$
BTC-only, 22-col	6	**0.989**	not significant, 0 indistinguishable pairs	$p \approx 0$

Table 4.2: Next-hour directional predictive signal (AUC, logistic regression, walk-forward test split) for the same six configurations.

Configuration	**regime_only AUC**	**features_only AUC**
Production (4-asset)	0.489–0.505	0.506–0.544
5-asset, 14-col	0.498–0.523	0.528–0.548
5-asset, 22-col	0.503–0.521	0.528–0.548
BTC-only, 14-col	0.494	0.542
BTC-only, 22-col	0.501	0.542

preferred for anything requiring a reproducible regime definition across refits, on the reasoning that a return-distinctness advantage that does not translate into predictive signal is worth less than reproducibility, but this is a judgment call between two real, opposing properties rather than a configuration that dominates on every axis.

4.1.2 Regime Characterization

Table 4.3 reports the fitted per-state statistics for the six-state, five-asset, 22-column configuration — the production regime model referenced throughout the remainder of this chapter and used by both the multi-agent and RL layers. Labels are generated automatically from each state's $\overline{\text{realized_vol}}$ (bucketed into five volatility tiers) and the sign of $\overline{\text{adx}}$ (trending versus range-bound), the same direction-agnostic classifier documented in Section 4.1.5; no directional ("bull"/"bear"/"crisis") language is used, for the reasons Section 4.1.4 establishes below.

Read purely as descriptive statistics, R0 and R5 resemble a natural short-or-avoid signal and R4 a natural long signal. Section 4.1.4 tests this reading directly at the level of individual price legs rather than accepting it at face value, and finds that it does not hold: these historical averages describe each state's unconditional population history over 2018–2025, not a leg-by-leg or forward-looking predictive property, and R4 in particular is dominant on *more* down-legs than up-legs despite carrying the highest historical Sharpe of any state.

Table 4.3: Per-regime fitted statistics, six-state five-asset 22-column configuration (pooled 2018–2025 training data). Sharpe, annualized return, and annualized volatility are historical population averages conditional on regime membership, not forward-looking predictions — see Section 4.1.4.

Regime	Label	Sharpe	Ann. Return	Ann. Vol	Time Share	Dominant A
R0	Very Low Volatility / Range-bound	−1.04	−34.8%	33.5%	15.2%	BTC
R1	Moderate Volatility / Trending	+1.11	+74.4%	67.0%	17.2%	LTC
R2	Low Volatility / Range-bound	−0.22	−14.7%	68.3%	18.8%	ETH
R3	Low Volatility / Range-bound	−0.26	−12.4%	46.9%	20.3%	XRP
R4	Elevated Volatility / Trending	+1.39	+140.6%	101.5%	19.4%	ETH
R5	Extreme Volatility / Trending	−0.44	−82.7%	189.8%	9.0%	XRP

4.1.3 Downstream Value: Position Sizing and Directional Conditioning

Three self-contained backtests — deliberately excluding the RL execution and multi-agent layers of Sections 3.6–3.7, so that the regime signal's standalone value could be isolated from anything a learned policy might extract from it — were run on the walk-forward validation (2024) and test (2025) splits of the five-asset panel, using the 22-column configuration.

Table 4.4: Regime-conditioned volatility-target position sizing versus buy-and-hold and continuous (regime-free) volatility targeting, five-asset equal-weight portfolio.

Split	buy_hold	vol_target (continuous)	regime_conditioned
2024 (validation)	**+1.203**	+0.353	+0.386
2025 (test)	**−0.131**	–0.343	–0.364

Buy-and-hold outperformed both sizing strategies in both periods, and regime-conditioned sizing was never significantly different from continuous, regime-free volatility targeting (block-bootstrap 95% confidence interval on the Sharpe delta, 1000 resamples over non-overlapping 168-hour blocks, crosses zero in both 2024 and 2025). Discretizing volatility into HMM regime buckets did not outperform simply dividing by trailing realized volatility directly for this sizing rule.

A second design tested whether regime could instead gate *when to trust* a directional signal, rather than only scale an always-long position: a 24-hour momentum rule (long or short by the sign of the trailing 24-hour return) was combined with a gate that permitted a position only when the current state had shown a positive Sharpe ratio for that same rule on training data, with the gate threshold decided from training data only in every case. A first pass, without a control, found this gate improving the momentum rule's Sharpe by roughly one full point in both periods (95% CI $[0.119, 2.094]$ in 2024, significant). A noise-control arm was then added — an independently simulated Markov chain using the real fitted transition matrix, giving it identical persistence and gating frequency to the

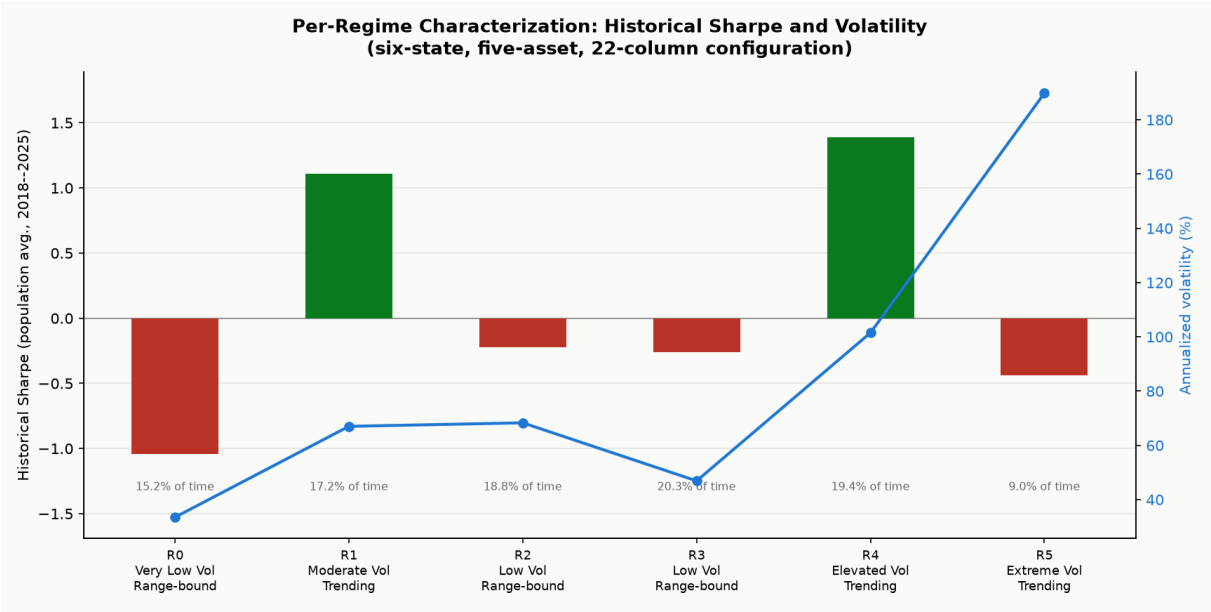


Figure 4.1: Per-regime historical Sharpe (bars, left axis) and annualized volatility (line, right axis), six-state five-asset 22-column configuration. Time-share annotations below each bar show the fraction of pooled 2018–2025 hours spent in that regime.

real regime label while carrying zero actual information about the asset's price path — and this changed the conclusion materially:

Table 4.5: Directional gating: real regime versus a matched noise control, momentum strategy, five-asset portfolio.

Split	**momentum_only**	**regime-gated**	**noise-gated**
2024 (validation)	–2.619	–1.488	**–1.050**
2025 (test)	–2.863	–1.998	–2.750

In 2024, the noise-gated control performed as well as, and by point estimate slightly better than, the real regime gate (regime-versus-noise 95% CI $[-1.721, 0.984]$, not significant), while both were significantly better than no gating at all. The underlying momentum rule is deeply unprofitable on its own (Sharpe -2.6 to -2.9 in both periods), and any gate that reduces net exposure to a structurally losing strategy mechanically improves its Sharpe ratio whether or not the gate carries genuine information — which is what the noise control demonstrates directly rather than leaving as a theoretical concern. 2025 shows a larger gap favoring the real regime gate (regime-versus-momentum CI $[-0.209, 2.162]$; noise-versus-momentum CI $[-1.134, 1.25]$), but neither period shows the real gate beating the noise control at conventional significance. Across all three backtest designs tested, standalone regime conditioning did not demonstrate a measurable, noise-control-surviving improvement in risk-adjusted return.

4.1.4 Regime Label Directional Fidelity: A Leg-Level Check

The three backtests above ask whether regime conditioning adds measurable value to a trading rule. A more basic question, prior to any trading rule, is whether a regime's own name describes what it actually does: does a state labeled "bull" predominate during

genuine price advances, and a state labeled "bear"/"crisis" during genuine declines? This was tested directly with a standard zigzag/peak-trough segmentation (a $\geq$15% threshold move, an objective, data-determined criterion, not tuned to the outcome) applied to daily close price for all five crypto5-panel assets over 2021 — a year containing both the panel's largest rally and one of its sharpest corrections. Each resulting leg (111 total across the five assets) was assigned the *dominant* (mode) regime label over its duration, and that label's class (bull-labeled, bear-labeled, or crisis) was checked against the leg's actual direction.

Table 4.6: Regime-label dominance by leg direction, five-asset panel, 2021 (111 legs total, zigzag threshold 15%).

Regime	**Label**	**Up-legs**	**Down-legs**	**% up**
R0	bear / drawdown	1	4	20%
R1	mild bull	6	3	67%
R2	mild bear / chop	21	17	55%
R3	low-vol bear	7	8	47%
R4	strong bull	13	19	41%
R5	crisis	6	6	50%

Overall match rate: **54/111 (48.6%)** — statistically indistinguishable from a coin flip. R0 and R1 show a genuine directional lean; R2 through R5 do not, and R4 — the state literally labeled *strong bull* — is dominant more often on down-legs than up-legs. This is not a rare miscall: two consecutive, unambiguous legs in the BTC series alone illustrate it directly. A -25.2% decline from 2021-01-08 to 2021-01-27 was dominated (52.5% of its hours) by R4; the very next leg, an $+89.1\%$ advance from 2021-01-27 to 2021-02-21, was dominated (67.0%) by R2, "mild bear/chop."

The cause is visible directly in the regime's own fitted feature means (Section 3.5.4): R4 has $\overline{\text{rsi}} \approx -0.005$ (direction-neutral) but $\overline{\text{realized_vol}} = +0.74$ and $\overline{\text{adx}} = +0.24$ (both clearly elevated) — R4 is not a bullish cluster, it is a *high-volatility, strongly-trending* cluster that fires on trend strength regardless of sign. Its historical Sharpe of +1.39, the statistic its name is derived from, is a population average over 2018–2025, a period in which crypto had more net-bullish high-volatility-trending hours than net-bearish ones; it describes the state's unconditional history, not a leg-by-leg predictive property, and is misleading if read as a forward-looking directional signal. This is consistent with, and considerably sharper than, every other finding in this section: the states are genuine volatility/trend-magnitude clusters (Section 4.1.1's distinctness result), not directional predictors, and their plain-language names should not be read as one.

4.1.5 Consequence: Removing Directional Language from Regime Descriptions (2026-09-01)

The zigzag result above is not only descriptive: the same sharpe-derived bull/bear/crisis text it discredits was, until this point, the literal string shown in the dashboard's regime legend and injected into every LLM agent's prompt as the qualitative description of current market conditions — exactly the kind of directional-sounding label the leg-level check just showed does not reliably correspond to actual price direction. This was corrected directly: a volatility/trend-strength classifier (built from $\overline{\text{realized_vol}}$ and $\overline{\text{adx}}$, both direction-agnostic by construction — ADX measures trend strength regardless of sign in

standard technical analysis) replaced the sharpe-derived bucketing in four places: the crypto5 production description generator and the legacy EURUSD/GOLD description generator in the dashboard backend (the latter fixed for consistency with the same anti-pattern, though not independently re-validated with an equivalent forex zigzag test), the static fallback dictionary, and — the change with actual behavioral reach — the description builder in the multi-agent context module (Section 3.6.3), which supplies the text every LLM agent reads as market character. A previously disconnected, hardcoded legend in the dashboard frontend was also wired to the same live source instead of a stale duplicate. Post-fix, e.g., what previously read *"Elevated Volatility / Strong Bull"* for R4 now reads *"Elevated Volatility / Trending"*. The full regime-quality audit (Sections 4.1.1, 4.1.6) was re-run afterward and confirmed byte-identical to its pre-fix output on every statistical section, since only description text was changed, not the underlying HMM decode.

A second, deferred issue was identified at the same time: the separate sharpe-bucketed classifier feeding `FundManagerAgent`'s conviction multipliers (Section 3.6.7) encoded the same directional assumption the zigzag test discredited — concretely, a bullish consensus occurring during R5 (crisis, empirically 50/50 directional) was unconditionally overridden to `hold` regardless of what the debate concluded, and R4 (empirically more down-legs than up-legs) carried a bullish-favoring conviction multiplier. Unlike the description fix above, this is trading logic rather than display text, so it was addressed as a separate, explicit change rather than folded into the same pass: the sharpe-sign classifier and its bullish/bearish penalty and override sets were removed, and conviction is now scaled by `classify_regime_volatility_level` — the same `realized_vol_mean`-derived, direction-agnostic dimension validated above — symmetrically for bullish and bearish calls, with no action-forcing override tied to an assumed direction. Verified directly: a bullish consensus during R5, which previously forced `final_action="hold"` irrespective of the debate's actual conclusion, now correctly returns `"long"` at reduced conviction (0.6×, reflecting R5's genuinely elevated volatility, not an assumed direction); R4 now scales bullish and bearish conviction identically (0.8× each) rather than favoring bullish. The trade-off is explicit: R0 and R1, the two regimes the zigzag test found to carry a real (if modest) directional lean, no longer receive any directional weighting either, since `classify_regime_kind` could not distinguish their validated lean from R2/R3/R4's discredited one without reverting to a per-regime-index table (rejected earlier in this project for going stale on every HMM refit). Full regression suite (50 tests across the agent and observation-builder test modules) passes unchanged after this edit.

4.1.6 Synthetic Recovery Summary

No mechanism tested moved recovery accuracy outside a narrow 0.15–0.20 band, including the configuration built to match production's real data scale and asset count directly, and the Bayes-optimal ceiling computed from the true generating parameters sits below every empirically achieved value in the table — confirming, as discussed in Section 3.5.8, that recovery accuracy at this task is bounded by genuine overlap between the six states' return distributions rather than by anything the estimation pipeline could be expected to improve with more data.

Table 4.7: Synthetic ground-truth recovery (Adjusted Rand Index) across every mechanism tested (Section 3.5.8).

Configuration	**ARI**
Single asset, 1 year, single seed	0.166
Single asset, 3 years, single seed	0.167
Single asset, 1 year, best-of-5-seed production procedure	0.156
5 independent synthetic assets pooled, 1 year each	0.163
5 correlated synthetic assets pooled (96% simultaneous co-movement)	0.176
5 assets pooled at production's real scale (≈246,000 rows)	0.162
Bayes-optimal classifier, true parameters, no estimation	**0.047**

4.2 Multi-Agent Deliberation Results

The multi-agent layer (Section 3.6) was audited end to end — prompt to final action, twenty real Mistral 7B calls, `BTC_USDT` at 2024-03-01, five purpose-built validation tests — rather than assumed to behave as its architecture description implies. The audit's central finding reframes the system's own description: it is not "multi-agent debate decides trades." Three agents (Macro, Technical, Sentiment) supply LLM-sourced directional opinions; one agent's dissenting direction is forced in code, not decided by its own model call; one agent (Risk) never calls an LLM at all; and the Fund Manager's actual decision — action, position size, conviction, dissent list — is deterministic arithmetic over the other agents' outputs, with its one LLM call producing only the rationale sentence, never the decision itself. The system is better described as a deterministic, risk-weighted consensus engine with LLM-assisted directional inputs.

4.2.1 Validation Battery: What the System Actually Does

Table 4.8: Multi-agent layer validation battery (20 real Mistral 7B calls, `BTC_USDT` 2024-03-01).

Test	**What it checks**	**Result**
Schema	Every call returns a valid, non-fallback response	6/6 agents pass
Null-regime	Swapping the real regime for a random one measurably changes output	3/3 directional agents pass
Aggregator-usage	Flipping one agent's thesis (no new LLM call) changes the Fund Manager's conviction and dissent list	Pass (0.81 → 0.51 conviction shift ob

The null-regime test confirms regime context is not silently discarded by the directional agents, and the aggregator-usage test confirms the deterministic vote genuinely reads every agent's output rather than hardcoding an answer — both properties that the architecture description alone would not establish.

4.2.2 Failure Modes Found, and Their Resolution

TechnicalAgent ignored obvious bearish price action; fixed and verified. On an adversarial case (RSI 12, strongly negative MACD, −6% return), the agent reported

"bullish, confidence 0.65," reasoning that the regime's qualitative description text ("high-volatility bull regime") supported a rebound. An ablation isolating the cause — rather than assuming weak reasoning in general — found the loaded "bull"/"bear" wording in the regime description text acting as a dominant lexical anchor for the 7B model: with that text present, 0/3 repeated runs on the adversarial case got the direction right; neutralizing the regime description entirely reached 1/3; stripping only the loaded wording while keeping the real regime label and probability vector (already confirmed informative by the null-regime test) reached 4/5. The fix builds the Technical agent's prompt from a context copy with the regime description replaced by a neutral placeholder, adds explicit ADX-conditioned interpretation rules for extreme RSI/MACD readings, and exposes `close_return` explicitly (previously silently truncated from the shared price-features block). Verified 5/5 correct on both bearish and bullish extremes post-fix, with the real, unmodified 2024-03-01 context re-checked to confirm normal-case output is unchanged.

Regime sensitivity is real but was initially not cleanly directional; this did not reproduce on the crypto-only panel. Forcing the Macro agent's regime block to the single best-Sharpe regime in the original four-asset dataset did not make it more bullish — output collapsed to neutral/0.50, the same response every synthetic regime (bull, bear, sideways, random) produced; only the *actual* regime, which happened to agree with the agent's own correlation data, produced a confident call. Re-running the identical synthetic-regime methodology after the migration to the five-asset crypto-only panel (Section 3.5.6) did not reproduce this collapse: the Macro agent scored 2/2 bullish on the bull variant, 2/2 bearish on the bear variant, and 2/2 neutral on the sideways variant. The Sentiment agent showed a milder version of the original inconsistency (1/2 bearish on the bear variant). Why the original finding no longer holds is not established — plausibly the crypto-only panel's regime descriptions carry more distinguishing information, plausibly the effect was always closer to the edge of reproducibility than the original characterization implied.

ContrarianAgent's dissent is intentionally not LLM-generated. Its `directional_bias` is forced to the opposite of consensus in code regardless of the model's own output; only confidence, reasoning, and key factors come from the model. This is a deliberate design choice — guaranteeing opposition for the Fund Manager to weigh — not a discovered defect, and is stated here so the system is not described as "a debating agent" without the caveat.

RiskAgent instantiates an LLM client it never calls. Harmless in practice (its GARCH-based volatility forecast and CVaR position caps are pure computation, Section 3.6.6), but worth correcting in any architecture diagram that implies otherwise.

Regime staleness could silently reach the prompt; fixed. The regime-context builder could serve a regime row up to twenty hours old on a data gap with no signal to the agent or the prompt. Fixed with an explicit staleness flag and gap-hours field, surfaced in the prompt when triggered; verified against both an exact-match timestamp (not flagged) and a known 20-hour gap (correctly flagged).

ContrarianAgent JSON parse failures had the wrong root cause identified initially. A prior hypothesis attributed roughly 40% of parse failures to output truncation at the token generation cap. A controlled repro found instead that all three test calls produced complete, well-formed JSON, and the actual failure was schema validation rejecting a reasoning field that exceeded its maximum length — the one prompt among the five agents' that never instructed the model to keep its answer short. Fixed by adding an

explicit length instruction to the prompt and adding a schema-generic recovery stage that clips any oversized field to its declared constraint via the schema's own field metadata, covering the same failure class for any other agent's field, not only this one.

4.2.3 Relationship to the Reported RL Policy

A direct search of the RL codebase for LLM usage returns exactly one substantive reference, which states outright that the debate system is never invoked during training or evaluation (Section 3.7). The checkpoint behind the RL results in Section 4.3 uses the raw-feature observation mode: none of its input dimensions are regime- or consensus-derived, and the debate cache's coverage of the RL validation window is on the order of one percent of validation steps even in observation modes that do carry a consensus slot. The multi-agent layer and the reported RL policy are two separately validated components, evaluated independently, not one integrated system, and are reported as such rather than implying the debate layer influenced the traded policy.

Migration and regression verification. Following the panel migration (Section 3.5.6), five real end-to-end debates (unmocked LLM calls) were run, one per crypto-panel asset, including three assets with no prior debate coverage. All five completed successfully; the Contrarian agent returned real historical-analogue evidence (31–66% historical loss rates) for every asset rather than the silent empty result it previously returned for the three newly added assets; the Risk agent produced five distinct real GARCH volatility forecasts; and the Macro agent's correlation matrix covered all ten asset pairs. Full regression after this and the parsing fix above: 83 of 83 tests passing in the affected suites, with four pre-existing failures elsewhere in the repository (dashboard API and RL-environment dimension drift) confirmed present before this work began and unrelated to it.

4.3 RL Execution Results

4.3.1 Controlled Ablation on the Crypto-Only Panel

Following the panel migration of the RL environment itself (asset sizing and portfolio/asset-identity state converted to instance-level parameters, Section 3.7.2), every lever proposed as a plausible fix for the policy's persistent failure to beat buy-and-hold was tested in isolation, one variable at a time, against the same BTC(+ETH) validation window. The buy-and-hold reference on this window has Sharpe **1.498** (91.8% total return).

Table 4.9: Controlled ablation, crypto-only panel, BTC(+ETH) validation window. Δ vs. buy-hold with 95% block-bootstrap CI (Section 3.8.3).

#	Configuration	Sharpe	Δ (95% CI)	Sig.	Trades
1	`raw_regime`, no fixes	–2.717	–4.300 [−7.103, −1.329]	worse	4,032
2	`raw`, no fixes	–1.211	–2.680 [−5.351, −0.044]	worse	6,019
3	`raw` + benchmark-relative reward + `min_hold_bars=24`	–0.515	–2.076 [−4.099, −0.048]	worse	5,991
4	(3) + behavior-cloning warm-start from a buy-hold teacher	**+1.461**	–0.035 [−0.140, 0.070]	no	181 (10 mean.)
5	`raw` + discrete 5-action space, `min_hold_bars=168`	–0.034	–1.643 [−3.714, 0.548]	incon- clu-sive	1,028

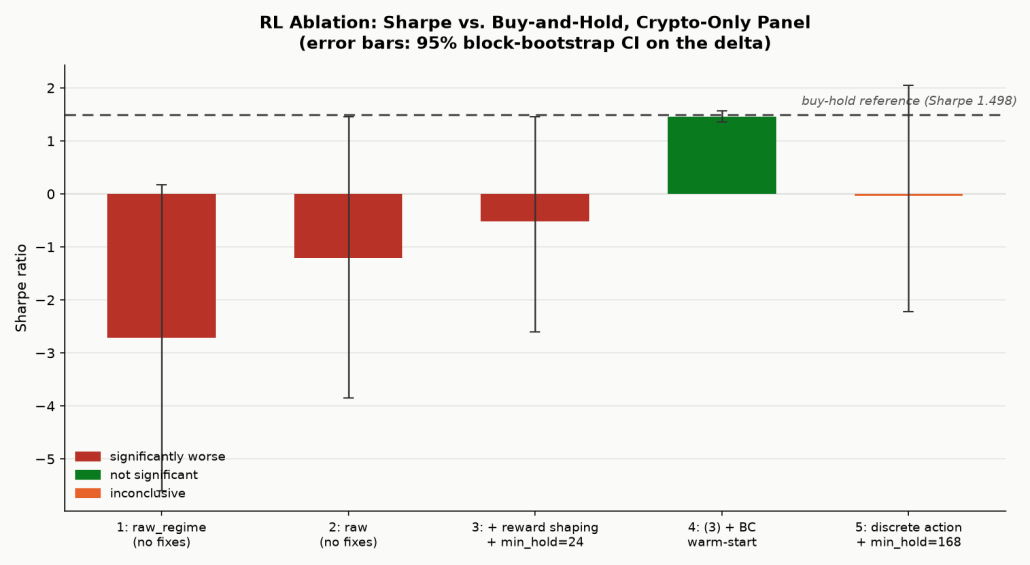


Figure 4.2: Absolute Sharpe ratio per ablation row against the buy-and-hold reference (dashed line, Sharpe 1.498). Error bars show the 95% block-bootstrap confidence interval on the Sharpe delta, re-centered onto the absolute Sharpe axis; bar color follows the significance verdict of Table 4.9.

Rows 1–2 isolate observation mode: dropping the regime-probability block improved Sharpe by roughly 1.5 points, consistent with a lookahead-bias defect identified independently (the HMM decode smooths over the entire concatenated sequence, so a regime probability at any training-split timestamp is influenced by data after it) — but both configurations remained significantly worse than buy-and-hold, so removing the leaky feature alone was not sufficient. Row 3 isolates reward shaping: benchmark-relative reward plus a longer minimum hold narrowed the gap further without resolving it, and the policy was still trading almost as often (5,991 trades). Row 4, behavior-cloning warm-start from a buy-hold teacher, is the only configuration not significantly different from buy-and-hold — but at only ten meaningful trades over a full year, and given a per-feature predictability check (Section 4.3.2) finding only a modest directional edge exists at all, this result cannot be confirmed as a genuine edge over its own teacher rather than a faithful reproduction of it; both readings are consistent with the data at this sample size. Row 5 tested whether the continuous Beta position head itself was driving excess trading; it was not — with a minimum hold seven times longer than row 3's, the discrete policy still traded roughly once every 8.5 days, almost exactly at the minimum-hold floor, and did not clear the pre-registered zero-Sharpe gate.

4.3.2 Predictability Ceiling Check

Before concluding the negative result reflects a structural limit rather than an unfound architecture, each of the crypto-panel's 22 features was tested alone (20 lags, ridge regression for magnitude R^2, logistic regression for direction, trained on data before 2024 and evaluated out-of-sample on 2024) against next-bar direction. Four of twenty-two features clear 52% directional accuracy (`rsi` 54.3%, `close_return` 53.4%, `obv_raw` and `macd_histogram` ≈53.1%); R^2 for return magnitude is approximately zero or negative for every feature tested. This rules out "there is genuinely nothing to find" as a complete

explanation — row 4's behavior-cloning-preserved edge is not obviously impossible given a real, if weak, 54% directional signal — without confirming that the RL formulation is leaving value on the table either: the edge is real but small, and every architectural lever tested against it either made results worse or reproduced its teacher without demonstrably exceeding it.

4.3.3 Final Decision on the RL Layer

Following a stopping rule set before the final ablation row ran, specifically to avoid open-ended iteration against a fixed project timeline: rows 1, 2, 3, and 5 are each either significantly worse than buy-and-hold or statistically inconclusive, and row 4 cannot be confirmed as a genuine edge over its own teacher at ten trades. Four independent architectural levers — regime conditioning, reward formulation, imitation warm-start, and action-space parameterization — were each tested in isolation, and none produced a confirmed, statistically defensible improvement over buy-and-hold. This was adopted as the RL layer's result for the project, not a configuration still being searched over, with three consequences: the dashboard (Section 3.9) ships with the multi-agent consensus and static execution path as its default signal source; RL execution remains available as an explicit, opt-in toggle carrying a warning about this section's finding; and this ablation table is reported as the project's controlled negative result on RL execution, evidenced end to end rather than asserted.

4.3.4 Native Weekly-Decision-Cadence Follow-Up (2026-08-31)

The results above train the policy to re-decide every hour. A follow-up experiment tested whether forcing a full week's commitment per decision — aggregating 168 hourly bars into a single decision, with the CVaR risk term scaled by $\sqrt{168}$ to remain comparable across horizons — changed the outcome, motivated by the ceiling check above finding only a weak *directional* edge that hourly rebalancing may be too fine-grained a horizon to extract.

Two engineering defects were found and corrected before any result from this experiment could be trusted: an annualization factor hardcoded for hourly returns, silently inflating an early result to an implausible Sharpe of 6.84 (a factor of $\sqrt{168} \approx 13\times$, traced and fixed by threading the evaluation horizon explicitly through every metrics call site); and an evaluation script that reloaded the in-memory end-of-training model rather than the best-by-validation-Sharpe checkpoint every other evaluation path in the project uses, which for one configuration alone produced a fourfold difference in reported Sharpe. Both fixes were verified by independent hand reconciliation before any number from this experiment was trusted further.

With both fixes in place, a smoke test (two configurations, one seed each) cleared pre-registered thresholds set before the test ran (Sharpe 2.198 and 2.209 respectively), triggering a full grid of twelve independently trained runs: four observation conditions (raw features; raw features plus realized volatility; raw features plus regime probability; raw features plus a placebo-regime noise control) $\times$ three seeds each.

None of the twelve 95% confidence intervals excludes zero (widest, $[-1.93, 3.10]$; the narrowest still crosses it). Two things are simultaneously true and neither cancels the other: twelve of twelve independently trained runs beat buy-and-hold at the point estimate, which has not occurred for any prior configuration across this project's history;

Table 4.10: Native weekly-cadence grid, BTC-only, validation year 2024. Sharpe recomputed with buy-and-hold aggregated to the identical weekly cadence (52 weekly observations); Δ from a block bootstrap (block size 4 weeks, 1000 resamples). Recomputed buy-hold reference: Sharpe 1.571. Full detail: `data/results/weekly_grid_reconciled.json`.

Seed	A (raw)	B (raw+vol)	C (raw+regime)	D (raw+placebo)
0	1.677 (Δ+0.26)	1.686 (Δ+0.20)	1.755 (Δ+0.43)	2.723 (Δ+1.50)
1	1.688 (Δ+0.31)	1.883 (Δ+0.45)	1.665 (Δ+0.40)	1.764 (Δ+0.57)
2	1.736 (Δ+0.10)	2.706 (Δ+1.42)	2.173 (Δ+0.84)	1.670 (Δ+0.01)

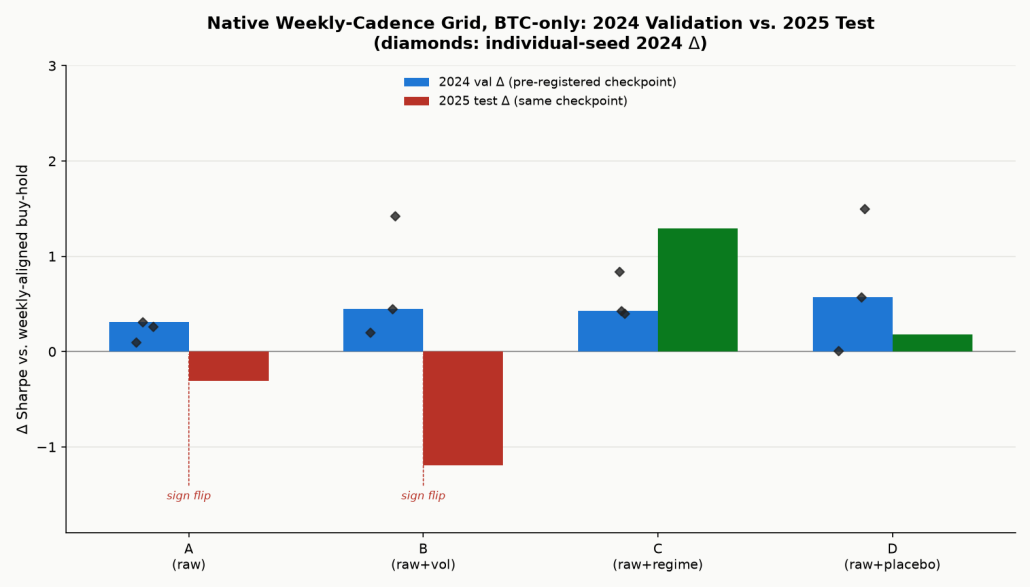


Figure 4.3: BTC-only weekly-cadence grid: the pre-registered checkpoint's Δ Sharpe vs. buy-hold on 2024 validation (blue) against the same checkpoint's Δ on 2025 test (red/green), from Table 4.11. Diamonds show the individual-seed 2024 Δ values of Table 4.10 as context for the spread the pre-registered checkpoint was drawn from. Conditions A and B flip sign between the two years; C and D do not.

and zero of twelve individually reach significance, with confidence intervals this wide at only 52 weekly observations per validation pass. A naive sign-test argument over twelve independent trials does not apply here, because the trials are not independent in the sense that argument requires: all twelve runs are evaluated against the identical single 2024 market window, so consistent point-estimate wins are equally consistent with "2024 favored a weekly-committed directional strategy generally" as with any genuine, architecture-specific skill. Condition D, an explicit noise control carrying no real regime information, performs in the same range as condition C, the real regime condition — consistent with every other regime-conditioning test in this chapter (Section 4.1.3) finding no measurable, noise-control-surviving value from regime conditioning.

At this point the result was a directionally consistent but statistically unconfirmed lead, pending out-of-window confirmation — resolved below.

2025 Test-Set Confirmation: the Lead Does Not Replicate

Following this project's established test-set discipline, one checkpoint per condition was selected *before* touching 2025 data, by median 2024 validation Sharpe among that condition's three seeds — not best-of-three, which would already be a mild form of cherry-picking on the validation set (A→seed 1, B→seed 1, C→seed 0, D→seed 1). This selection was fixed in the evaluation script before it was ever run against 2025, and the script was run exactly once.

Table 4.11: 2025 test-set pass, pre-registered checkpoints, one per condition. Δ vs. buy-hold; recomputed buy-hold weekly Sharpe on 2025 is −0.291 (−15.9% return), against +1.571 on 2024 — a materially different market year.

	A (raw)	**B (raw+vol)**	**C (raw+regime)**	**D (raw+placebo)**
2024 val Δ	+0.31	+0.45	+0.43	+0.57
2025 test Δ	**−0.31**	**−1.19**	+1.29	+0.18
Significant?	no	no	no	no

Conditions A and B flipped sign entirely — profitable relative to buy-hold in 2024, unprofitable relative to it in 2025 — consistent with the 2024 result reflecting conditions favorable to any weekly-committed directional strategy that particular year, rather than a robust cadence-specific edge, exactly the risk raised when the validation grid was first reported (Section 4.3.4). Condition C is the one positive point estimate, and the largest delta of the four, but its 95% CI is wide and crosses zero $[-1.709, 4.311]$, and a single pre-registered checkpoint on test cannot separate a genuine regime-conditioning benefit from that particular seed's idiosyncrasy. Condition D, the noise control, sits close to buy-hold and is not clearly separated from C either.

This closes the investigation as not confirmed. It does not overturn Section 4.3.3's final decision on the RL layer; it adds one further out-of-window data point consistent with it.

Crypto5 Panel Extension: the Null Result Generalizes (2026-09-01)

The BTC-only result above leaves one methodological concern unaddressed: 2024 was not equally favorable across assets (BTC bull, ETH mostly consolidating, BNB up on low volatility, LTC sideways-then-up, XRP up late), so a result benchmarked on BTC alone risks mistaking a favorable tailwind for genuine performance. This was tested directly by extending the same native-weekly-cadence design to the full crypto5 panel: a pilot grid (5 assets × the same 4 conditions, 1 seed each, 20 independently trained policies) reconciled with the identical weekly-aligned buy-hold and block-bootstrap methodology as the BTC-only result.

One consequential defect was found and fixed before trusting any cross-asset result: `evaluate_ppo()`'s validation-environment builder silently defaulted to a hardcoded BTC-only asset list whenever a caller omitted an explicit one — dormant until this point because every prior caller's configuration either was BTC-only or included BTC. Left uncaught, every non-BTC cell would have either evaluated a correctly-trained policy against BTC's own validation data or crashed outright, exactly the class of silent asset-mismatch defect this project has encountered more than once. Fixed by threading the configured asset list through both the validation-environment builder and the buy-hold baseline call, which carried the identical hardcoded default.

2024 validation, reconciled: 18 of 20 cells positive at the point estimate, but only **one of twenty reaches significance — and it is condition D, the placebo/noise-control regime, not C, the real regime.** If regime conditioning carried genuine value, C should be the cell separating from noise; instead the opposite occurred on the one cell that cleared significance at all.

Long-run diagnostic and 2025 out-of-window check, single pass per checkpoint, no re-runs: each of the 20 checkpoints was walked continuously from 2024-01-01 through the last available bar (the crypto5 panel's aligned data ends 2025-12-31; a longer horizon is not reachable with this data), and the 2025-only portion of that same trajectory — not a separate re-run — was extracted as the out-of-window check.

Table 4.12: 2025-only Δ vs. buy-hold, extracted from a single continuous 2024–2025 walk-forward pass per checkpoint, crypto5 panel, 1 seed. 0 of 20 cells significant.

	A (raw)	**B (raw+vol)**	**C (raw+regime)**	**D (raw+placebo)**
BTC	+1.27	–0.46	+0.15	+0.60
ETH	**–0.39**	**–0.43**	**–1.68**	**–0.35**
BNB	–0.89	–1.18	+0.65	–1.30
LTC	+0.55	+0.86	+0.22	+0.57
XRP	+2.41	–0.08	+0.01	+1.18

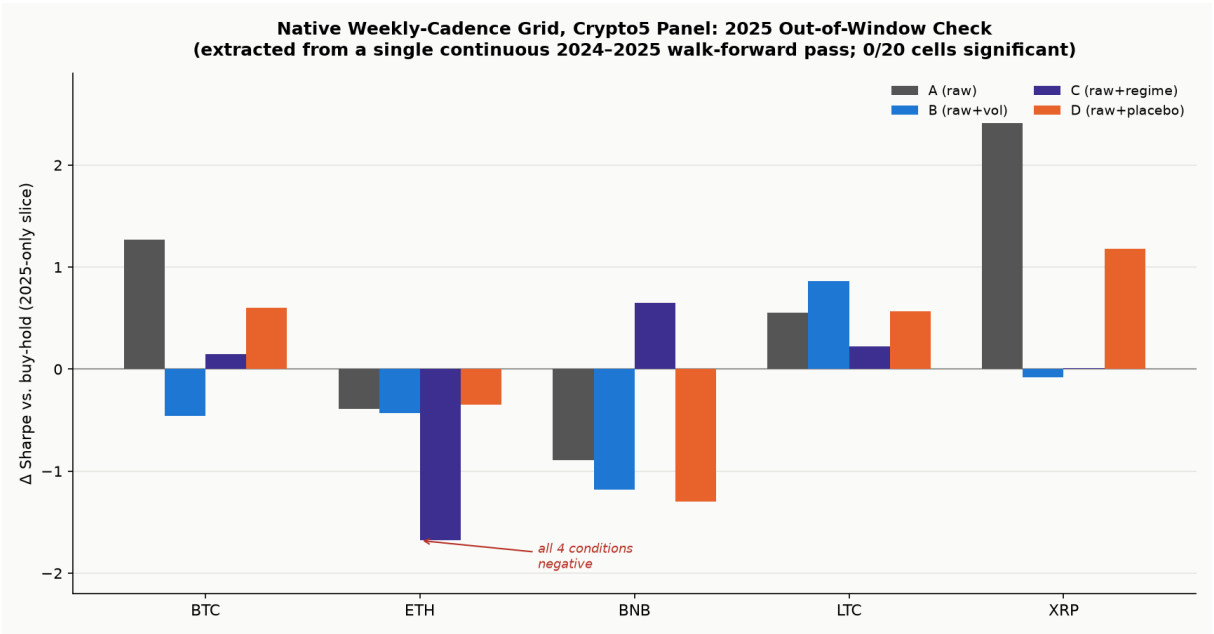


Figure 4.4: Crypto5 panel extension, 2025-only Δ Sharpe vs. buy-hold per asset and condition (Table 4.12). ETH is the one asset where all four conditions, including both the real regime condition (C) and its noise control (D), lose to buy-hold.

The 2024 pattern (18/20 positive) weakens to 11/20 positive on the 2025 out-of-window check — close to a coin flip — and **ETH reverses completely: all four conditions lose to buy-hold in 2025**, having looked positive in the 2024 pilot. This is the identical favorable-year-artifact signature already found for BTC alone, now independently reproduced on a second asset. Condition C shows no separation from D on either split, on any asset.

Final verdict: this closes the crypto5 extension as consistent with, not contradicting, every prior result on regime conditioning in this project. Across 40 total comparisons (20 whole-period, 20 test-slice) none reach significance. Having now replicated independently on BTC alone, on the full five-asset panel's 2024 validation, and on the 2025 out-of-window check, further seeds or grid expansion were judged unlikely to change this conclusion and were not pursued.

Chapter 5

Conclusion

Chapter 4 reported three independently audited bodies of evidence: the regime-detection layer's statistical validity and downstream value (Section 4.1), an end-to-end behavioral audit of the multi-agent deliberation layer (Section 4.2), and a controlled ablation of the RL execution layer culminating in a final decision on that layer (Section 4.3). An earlier version of this chapter closed out only the regime layer, on the reasoning that the multi-agent and RL audits were not yet complete; both have since been completed and reported in full, so this chapter now synthesizes all three into one project-level verdict, rather than leaving two of the three layers without a stated conclusion.

5.1 Regime Detection: Findings

5.1.1 What Was Found

Two engineering defects predating this project's audit work were found and corrected: a subsampling procedure in the BIC-based state-count search that fit the model on randomly reordered, non-contiguous rows, corrupting the estimated persistence of every candidate state count; and the absence of any sequence-boundary handling in a pipeline that concatenates multiple assets' histories into one matrix, which had every fit and decode silently treating one asset's final observation as immediately preceding another asset's first. Fixing the second exposed that no seed-sensitivity check had ever existed in the codebase; adding one retroactively showed the production four-asset model's regime partition was not robust to EM initialization (mean pairwise ARI 0.193 across five seeds), a previously unmeasured property of a model that had already been in use.

Independently, a raw-feature variant of the detector was found to have collapsed onto asset identity rather than market regime on the original four-asset panel, traced to a specific, identifiable property of the forex/commodity data source (a literally constant, not merely low-variance, volume-derived feature) dominating an unwhitened Gaussian mixture. This motivated a full panel revision to a crypto-only, five-asset universe, which was verified, not assumed, to resolve the defect (maximum single-asset state share falling from 0.97–0.99 to 0.234) and was carried through the complete engineering and statistical validation battery described in Chapter 3: pipeline validation, empirical state-count rechecking at $n \in \{3, \ldots, 8\}$, the six-configuration statistical comparison of Section 4.1.1, three independent downstream backtests with an explicit noise control (Section 4.1.3), a leg-level directional fidelity check (Section 4.1.4), and a purpose-built synthetic ground-truth validation methodology (Section 3.5.8).

Three results from that battery are robust across every configuration tested and are the ones this layer's conclusion rests on. The discovered regimes are genuine, statistically significant volatility clusters ($p \approx 0$ on realized-volatility distinctness, six for six configurations, no exceptions). Regime conditioning did not demonstrate measurable value in any of the three standalone downstream tests attempted — volatility-target position sizing, cross-asset systemic allocation, and noise-control-verified directional gating — nor in next-hour directional prediction, where the regime label alone was consistently at chance level. And the plain-language directional names originally attached to each regime ("bull", "bear", "crisis") were themselves shown to be unreliable at the leg level (48.6% match rate, indistinguishable from a coin flip), which is why they were removed from every downstream consumer of regime text, including the multi-agent prompt and the Fund Manager's conviction logic (Section 4.1.5).

5.1.2 Interpretation

These results are not in tension with one another. A signal can be statistically real without being independently actionable by a simple rule: the synthetic validation work (Section 3.5.8) traced the ceiling on per-timestep regime recovery to genuine overlap between the six states' return distributions at hourly resolution — a Bayes-optimal classifier given the true generating parameters could not exceed an Adjusted Rand Index of 0.047, below what the actual estimated pipeline achieves by aggregating information over rolling windows and exploiting the fitted model's own transition persistence. The states are separable enough for a well-specified model to reproducibly agree on where they are (once the feature representation and asset panel were fixed), and volatile enough to be statistically real by any conventional test, without necessarily being separable enough, at single-hour resolution, to hand a simple rule the information it would need to act on them directly.

This bears directly on the core hypothesis motivating this project, stated in Section 2.3 and in the formal problem statement of Section 1.3.3: that explicit regime conditioning improves trading outcomes relative to regime-agnostic learning. The regime-layer evidence alone neither confirms nor refutes that hypothesis at the level it was originally stated — it refutes a narrower one, that a simple rule conditioned on the regime label outperforms an equivalent rule that is not, under three independent formulations and a genuine noise control. The hypothesis as originally stated is a claim about a *learned* policy's or a *reasoning* system's ability to extract value from regime conditioning that a hand-specified rule cannot; testing that claim properly requires exercising the multi-agent and RL layers, which Sections 5.2–5.3 below now do.

5.2 Multi-Agent Deliberation: Findings

The audit reported in Section 4.2 reframes the system's own architectural description. It is not, as Section 3.6 might imply in isolation, "five agents debate and decide." Three agents (Macro, Technical, Sentiment) supply LLM-sourced directional opinions; the Contrarian agent's dissenting direction is forced in code by design, not decided by its own model call; the Risk agent never calls an LLM at all, producing its output from GARCH volatility forecasting alone; and the Fund Manager's actual decision — action, position size, conviction, dissent list — is deterministic arithmetic over the other agents' outputs, with its own LLM call producing only the natural-language rationale, never the decision

itself. The system is more accurately described as a deterministic, risk-weighted consensus engine with LLM-assisted directional inputs, and is reported as such rather than as an autonomous multi-agent debate.

Within that corrected description, the layer passes every test in its validation battery (Table 4.8): regime context measurably changes directional agents' output rather than being silently discarded, and the Fund Manager's deterministic vote genuinely reads every agent's output rather than hardcoding an answer. Five concrete failure modes were found and fixed during the audit, the most consequential being a Technical agent that anchored on the loaded "bull"/"bear" wording in the regime description text strongly enough to call a −6%-return, RSI-12 case "bullish" — fixed at the root by the same directional-language removal that Section 4.1.5 applied project-wide, and independently verified 5/5 correct post-fix on both bearish and bullish adversarial cases.

Crucially, the multi-agent layer and the RL policy behind this project's headline execution results are two separately validated components, not one integrated system: the RL checkpoint reported in Section 4.3 runs in a raw-feature observation mode with no regime- or consensus-derived input dimensions at all, and the debate system is never invoked during RL training or evaluation (Section 4.2.3). This report accordingly does not, and cannot, claim that the multi-agent layer's reasoning influenced the traded policy evaluated in Section 5.3; the two layers' conclusions are independent findings about independent components, not two halves of one measured system. A standalone backtest of the multi-agent layer's own trading value — running its consensus output as a rule-based signal, the same way Section 3.8.2's `agents_no_regime` variant already does at a coarse level — is the natural remaining test of this layer specifically and is noted as an open item in Section 5.5.

5.3 RL Execution: Findings

Section 4.3.1's controlled ablation tested four independent architectural levers against the policy's persistent failure to beat buy-and-hold on the crypto-only panel — observation mode (regime-probability block present or absent), reward formulation (benchmark-relative reward with a longer minimum hold), imitation warm-start (behavior cloning from a buy-hold teacher), and action-space parameterization (discrete vs. continuous position sizing) — one variable at a time, against a shared buy-and-hold reference (Sharpe 1.498). None produced a confirmed, statistically defensible improvement: three of the five configurations tested were either significantly worse than buy-and-hold or statistically inconclusive, and the one configuration not significantly different from buy-and-hold (behavior-cloning warm-start) traded only ten times over a full year, too few trades to distinguish a genuine edge over its own teacher from a faithful reproduction of it. The predictability ceiling check (Section 4.3.2) ruled out "there is nothing to find" as a complete explanation — a real, if weak, 52–54% directional edge exists in a handful of raw features — without establishing that any tested RL formulation was leaving that value on the table unexploited.

The native weekly-decision-cadence follow-up (Section 4.3.4) retested the same question at a coarser, week-level commitment horizon, motivated by the ceiling check's finding that the directional edge, where it exists at all, is weak. Twelve independently trained BTC-only configurations all beat buy-and-hold at the point estimate on 2024 validation data, the first time this had occurred anywhere in the project, but none reached statistical significance, and Condition A/B's 2024 lead reversed sign entirely on 2025 test data

(Section 4.3.4) — consistent with 2024 having favored any weekly-committed directional strategy that year, not a genuine cadence-specific edge. Extending the same design to the full five-asset panel (Section 4.3.4) reproduced this pattern independently on a second and third asset (ETH reversing completely between 2024 and 2025; only 1 of 20 cross-asset cells reaching significance on 2024 validation, and that one cell being the noise-control placebo condition rather than the real regime condition) and found no cell, on either the whole-period or 2025-only comparison, where the real regime condition separated from its matched noise control.

Final decision, adopted for this project rather than left open pending further search: across the original ablation, the predictability ceiling check, and two independent weekly-cadence follow-ups (BTC-only and five-asset), no tested configuration of the RL execution layer produced a confirmed improvement over buy-and-hold, and no tested configuration that included real regime information outperformed a matched noise control carrying none. This decision has three concrete consequences already reflected in the shipped system (Section 3.9): the dashboard defaults to the multi-agent consensus and static execution path as its signal source; RL execution remains available only as an explicit, opt-in toggle carrying a warning that references this section; and this ablation record, not a single headline Sharpe number, is reported as the project's result on RL execution.

5.4 Overall Verdict

Section 1.3.3 posed the project's success criterion explicitly: not trading performance alone, but whether regime conditioning demonstrably improves outcomes relative to regime-agnostic baselines. That question was tested three times, by three structurally different mechanisms, each with its own noise control: a rule-based signal conditioned directly on the regime label (Section 4.1.3), a multi-agent LLM reasoning layer conditioned on regime context (Section 5.2), and a learned RL policy conditioned on the regime embedding (Section 5.3). **In no case did regime conditioning demonstrate a confirmed, noise-control-surviving improvement.** This is a consistent finding across independent mechanisms and independent evaluation periods, not an isolated negative result from one experiment.

This does not mean the regime layer found nothing. The discovered states are genuine, statistically significant, reproducible volatility clusters — a real structural property of the underlying markets, confirmed by seed-stability analysis, ANOVA distinctness testing, and a purpose-built synthetic ground-truth methodology with a Bayes-optimal ceiling below the pipeline's own achieved accuracy. What the evidence gathered here does not support is the stronger claim that this real structure, once discovered, is straightforwardly actionable through any of the three specific mechanisms this project built to exploit. The project's honest empirical answer to its own central hypothesis is therefore negative on actionability, not negative on existence: regimes are real; using them, at hourly-to-weekly decision granularity on a five-asset crypto panel, through a rule, an LLM debate, or an RL policy, was not shown to help.

A negative result reached this rigorously is itself a contribution distinct from the specific hypothesis it tests. Every layer of this project underwent an audit that found and fixed real defects predating the audit itself — a corrupted BIC search and a missing sequence boundary in the regime layer, a lexical-anchoring bug and silent JSON-parsing failures in the agent layer, a lookahead-biased observation feature and two independent

evaluation bugs (a hardcoded annualization factor, a wrong-checkpoint reload) in the RL layer — and every headline number in Chapter 4 is reported alongside the noise control, bootstrap confidence interval, or out-of-window replication that either supports or undermines it, rather than as an isolated point estimate. That evaluation discipline, applied consistently across three architecturally very different layers, is what makes the negative result above trustworthy rather than merely an absence of a positive one.

5.5 Limitations and Open Items

Several findings and design questions were deliberately left unresolved during this project rather than silently dropped, and are recorded here for that reason.

Regime layer. The legacy four-asset model's seed instability was diagnosed but never corrected — only the revised five-asset panel received the stability improvement, since the four-asset panel was being superseded rather than maintained. The Gaussian emission distribution's underestimation of extreme moves, surfaced by the synthetic validation's tail-probability cross-check, was identified but not addressed with an alternative emission distribution; this diagnosis was made on the synthetic *calibration* model rather than confirmed against the production fit's own real-data residuals, so its practical impact on the deployed HMM remains unquantified. The encoder's own refinement experiments (last-timestep pooling; CPC-style future-window pairing in place of same-window augmentation, Section 3.4.4) were still in progress at the time of writing and did not reach a conclusive result either way.

Multi-agent layer. A standalone backtest isolating the multi-agent consensus's own trading value — independent of the RL layer entirely, since Section 5.2 established the two are not integrated — is the most direct remaining test of this layer and was not carried through to a full, statistically-tested result in this report. The Bull/Bear Researcher split proposed as a symmetric alternative to the single Contrarian agent (mirroring TradingAgents' research-team structure, Section 2.4.1) remains an unimplemented design proposal.

RL layer. Most ablation cells in this report are single-seed; the weekly-cadence grid is the only three-seed result, and even there, confidence intervals at 52 weekly observations per validation pass are wide enough that a substantially larger multi-seed campaign would be needed to meaningfully narrow them. The four architectural levers tested in Section 4.3.1 were tested one at a time rather than in combination, so interaction effects between, for instance, reward shaping and action-space parameterization were not explored.

Scope. No component of this evaluation used real capital; all results are backtested or paper-traded, and the live dashboard (Section 3.9) exists to make the system auditable, not to constitute a live-trading performance claim. Given the consistency of the negative result across every mechanism tested, the most defensible direction for further work is not a fifth attempt at conditioning a trading rule on this regime signal, but either a fundamentally different representation of regime (addressing the per-timestep overlap the synthetic validation traced the ceiling to) or a redirection of effort toward the parts of the pipeline — data quality, evaluation discipline, and auditability — that this project's own audits repeatedly found to be where the real, fixable defects actually were.

Appendix A

Appendix

A.1 Synthetic Regime Ground-Truth Validation: Full Derivation

This appendix gives the full construction and calibration of the synthetic regime-switching process summarized in Section 3.5.8, used to measure regime-recovery accuracy against a known ground truth — something no validation against real market data can provide, since the true underlying regime process, if one exists, is never observed there.

A synthetic regime-switching process was constructed and calibrated to the real, already-fitted five-asset model: a six-state Markov-modulated geometric Brownian motion using the fitted transition matrix and each state's empirical annualized return and volatility directly as the generating parameters, so that the synthetic process matches, by construction, what the fitted model itself believes the data-generating process looks like.

Two independent discretization schemes were implemented to simulate price paths from this process, so that any recovery result could be checked for sensitivity to the simulation method itself rather than trusting a single generator. Level-space Euler–Maruyama and Milstein schemes were both implemented for the underlying SDE, and an empirical convergence-order comparison was attempted between them, holding the regime fixed to isolate discretization error from regime-switching noise; at the hourly step size and volatility levels relevant to this project, the theoretically expected $O(\Delta t)$ discretization bias between the two schemes (≈ 0.007 in variance units) was found analytically to sit roughly 28 times below the Monte Carlo noise floor achievable at a feasible sample size (200,000 paths, noise floor ≈ 0.19), so no resolvable difference between the two schemes was expected or found — a legitimate methodological conclusion (Euler–Maruyama's multiplicative discretization is already highly accurate for GBM at these parameters) rather than an inconclusive test. A second, non-SDE generator was implemented independently: a regime-switching trinomial tree with Cox–Ross–Rubinstein-style fixed jump sizes per state, whose three jump probabilities are solved numerically (`scipy.optimize`, Nelder–Mead) to match each state's target mean and variance exactly, subject to summing to one — a genuine small calibration problem rather than a closed-form shortcut.

Three further numerical cross-checks validated the calibrated process's implied distributional properties against independent methods before it was used for anything else. A Crank–Nicolson finite-difference solution of the Kolmogorov forward equation for log-price, regime held fixed, was checked against the closed-form Gaussian solution available in that special case; an initial implementation returned a variance exactly double the closed-form value on every tested state (a coefficient error — the Crank–Nicolson $\theta = 0.5$

averaging factor had been omitted from the diffusion coefficient), and once corrected, the finite-difference solution matched the closed form to 0.04% relative error uniformly across all six states. The regime-weighted mixture density's tail probabilities were computed by direct numerical integration (`scipy.integrate.quad`) as a semi-analytical alternative to simulation. Finally, the exact discrete-time characteristic function of the cumulative return over a fixed horizon was derived via a transfer-matrix recursion over the six states (not an approximation) and inverted by FFT to obtain the model-implied 24-hour return density; the resulting 24-hour variance (0.00103) came out roughly half of what treating each hour's regime draw as independent would predict (0.00206), consistent with the fitted transition matrix's high self-persistence (∼93–96% per hour) making a 24-hour window far more likely to stay within one regime than an i.i.d. approximation would suggest.

With the generator validated, the central question — does the actual feature-engineering and HMM-fitting pipeline (Sections 3.3.2, 3.5.3–3.5.4) recover the true regime label from data it did not see the labels for — was tested by simulating synthetic OHLCV series with a known regime at every hour, running them through the unmodified production feature pipeline and a fresh single-asset HMM fit, and scoring the recovered labels against the known truth with Adjusted Rand Index and a Hungarian-matched classification accuracy (`scipy.optimize.linear_sum_assignment` on the confusion matrix, which is invariant to the arbitrary label ordering EM produces). Five candidate explanations for the resulting recovery level were tested directly rather than assumed, each isolating one variable: more calendar time per path (one year versus three, single asset), the actual best-of-five-seed production fitting procedure in place of a single seed, pooling multiple statistically independent synthetic assets, pooling multiple assets with a strongly and directly verified cross-asset regime correlation (a shared latent market-regime process each asset's own regime follows with high, controlled probability), and matching production's real training-set scale exactly (≈246,000 pooled rows). None of the five moved recovery accuracy outside a narrow band (Adjusted Rand Index 0.15–0.20 across every configuration tested, including the one built to match production's real scale and asset count directly), ruling out data quantity, cross-asset correlation, and seed count as explanations in this setting.

The eventual explanation was obtained directly rather than inferred: a Bayes-optimal classifier constructed from the *true* generating parameters — no feature engineering, no HMM estimation, the best possible single-observation classification given perfect knowledge of the answer — achieved an Adjusted Rand Index of only 0.047, below what the full estimated pipeline achieved in every configuration tested. Inspecting the calibrated parameters directly explains why: the six states' hourly log-return means are all within an order of magnitude of zero and of each other, and even the volatility differences that do separate the states overlap substantially when only one hourly observation is available. The states are differentiated almost entirely by volatility level, and distinguishing six heavily overlapping, near-zero-mean Gaussians from single noisy draws is close to an irreducible classification problem regardless of how well its parameters are estimated. This also resolves an apparent tension with the seed-sensitivity results of Section 3.5.5: seed agreement measures whether independent EM restarts converge to the *same estimated parameters*, which more data does improve, while the synthetic recovery test measures whether the resulting model correctly labels each individual hour, which is bounded by class overlap and is a different property that more data does not fix. A model can be perfectly reproducible across refits while remaining noisy at the per-timestep level; the two findings are consistent rather than contradictory once the two questions are separated.

A.2 Implementation Note: A Multiprocessing Deadlock

While parallelizing the five-seed HMM refit of Section 3.5.5 across CPU cores, a silent hang — not an incorrect result — was traced to Python's default `fork`-based process pool deadlocking completely when launched immediately after a BIC-search fit large enough to trigger multi-threaded BLAS execution in the parent process. Forking while another thread holds a BLAS-internal lock duplicates only the calling thread, so the lock is never released in the child and the first BLAS call inside any worker hangs indefinitely; five worker processes were observed sitting at under 1% CPU utilization for over fifteen minutes before the defect was diagnosed. Switching the process pool to the `spawn` start method, which gives every worker a genuinely fresh interpreter with no inherited thread state, resolved it; the fix was verified by confirming all five workers reached the expected 97–98% CPU utilization and by the synthetic sanity check of Appendix A.1 run on pure random noise, which correctly returned near-chance seed agreement once the pool was actually computing.

Bibliography

[1] Pete Chapman, Julian Clinton, Randy Kerber, Thomas Khabaza, Thomas Reinartz, Colin Shearer, and Rudiger Wirth. Crisp-dm 1.0: Step-by-step data mining guide. Technical report, SPSS Inc., 2000.

[2] James D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

[3] Yang Li, Yangyang Yu, Haohang Li, Zhi Chen, and Khaldoun Khashanah. Trading-gpt: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance. *arXiv preprint arXiv:2309.03736*, 2023.

[4] Market regime detection using hidden markov models. QuestDB Glossary. Accessed 2026.

[5] Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*, 2020.

[6] Jean Ndoutoumou, Zining Yin, and Xiaochang Cheng. Hmm-based market regime detection with rl for portfolio management. In *2025 IEEE 11th International Conference on Intelligent Data and Security (IDS)*, page 67, 2025.

[7] Wentao Zhang et al. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. 2024.

[8] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[11] Bryan Lim, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.

Résumé

Ce projet de fin d'études propose MARTA (*Multi-Agent Regime-Aware Trading Architecture*), une architecture de trading algorithmique multi-couches conditionnée par la détection explicite de régimes de marché. Le système combine : (1) une ingénierie de variables techniques et statistiques calculées sur cinq actifs crypto à résolution horaire (BTC, ETH, BNB, LTC, XRP) ; (2) un détecteur de régime fondé sur un modèle de Markov caché gaussien, identifiant des états de volatilité statistiquement distincts et reproductibles ; (3) un système de délibération multi-agents reposant sur un grand modèle de langage (cinq agents spécialisés — macro, technique, sentiment, risque, contrariant) produisant un consensus pondéré conditionné par le régime détecté ; (4) un agent d'exécution entraîné par apprentissage par renforcement (PPO) traduisant ce consensus en décisions de position ; et (5) un tableau de bord de supervision en temps réel.

L'hypothèse centrale du projet — à savoir que le conditionnement par régime améliore les résultats de trading par rapport à des approches agnostiques au régime — a été testée rigoureusement à travers trois mécanismes structurellement indépendants (une règle déterministe, un raisonnement multi-agents par LLM, et une politique de RL apprise), chacun évalué face à son propre contrôle de bruit. Dans aucun des trois cas cette amélioration n'a pu être confirmée statistiquement, bien que les régimes découverts se révèlent être des structures de volatilité réelles, statistiquement significatives et reproductibles. Ce résultat négatif, obtenu avec une discipline d'évaluation rigoureuse (tests de significativité, intervalles de confiance par bootstrap, réplication hors échantillon), constitue en lui-même une contribution méthodologique du projet, indépendamment de l'hypothèse de recherche initiale.

Mots-clés : trading algorithmique, détection de régime de marché, modèle de Markov caché, agents LLM, apprentissage par renforcement, explicabilité, validation statistique.

Abstract

This final-year project presents MARTA (Multi-Agent Regime-Aware Trading Architecture), a layered algorithmic trading system conditioned on an explicit, learned representation of market regime. The pipeline combines: (1) engineered technical and statistical features computed on five hourly-resolution crypto assets (BTC, ETH, BNB, LTC, XRP); (2) a Gaussian Hidden Markov Model regime detector identifying statistically distinct, reproducible volatility states; (3) a five-agent large-language-model deliberation system (macro, technical, sentiment, risk, and contrarian perspectives) producing a regime-conditioned, risk-weighted consensus; (4) a PPO-trained reinforcement learning agent translating that consensus into position-sizing decisions; and (5) a real-time monitoring dashboard.

The project's central hypothesis — that explicit regime conditioning improves trading outcomes relative to regime-agnostic baselines — was tested through three structurally independent mechanisms (a deterministic rule, LLM-based reasoning, and a learned RL policy), each evaluated against its own noise control. In no case did regime conditioning demonstrate a statistically confirmed improvement, although the discovered regimes are shown to be genuine, statistically significant, and reproducible volatility structures. This rigorously obtained negative result — supported by significance testing, bootstrap confidence intervals, and out-of-sample replication across independent evaluation periods — is itself reported as a methodological contribution of the project, independent of the original research hypothesis.

Keywords: algorithmic trading, market regime detection, hidden Markov model, LLM agents, reinforcement learning, explainability, statistical validation.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454